



دانشگاه صنعتی اصفهان
دانشکده مهندسی مکانیک

پروژه درس طراحی سیستم های کنترلی
شبیه سازی و کنترل سیستم دو درجه آزادی آونگ معکوس

استاد راهنما
دکتر محمدجعفر صدیق

محققین

علی نصر ۸۸۲۶۳۹۳

عمادالدین پی سپار ۸۸۱۲۲۸۳

خرداد ۹۲

فصل اول: مقدمه	۴
مشخصات مسئله	۴
فصل دوم: مدل سازی سیستم	۶
معادلات سیستم system equations	۶
تابع تبدیل Transfer Function	۷
معادلات فضای حالت State-Space	۸
فصل سوم: نمایش در نرم افزار MATLAB	۹
تابع تبدیل Transfer Function	۹
معادلات فضای حالت State-Space	۹
فصل چهارم: تجزیه و تحلیل سیستم	۱۲
پاسخ ضربه سیستم حلقه باز	۱۲
پاسخ پله سیستم حلقه باز	۱۴
فصل پنجم: طراحی کنترل کننده	۱۶
طراحی کنترل کننده با $K=0$ مربوط به دینامیک سیستم	۱۶
طراحی کنترل کننده با $K=1$ مربوط به دینامیک سیستم	۱۸
طراحی کنترل کننده با $K=100$ مربوط به دینامیک سیستم	۲۲
فصل ششم: روش فضای حالت برای طراحی کنترل	۲۵
قطب های حلقه باز Open-loop poles	۲۶
روش مقررات درجه دوم خطی (LQR)	۲۷
روش استقرار قطب مقاوم place	۳۰
فصل هفتم: طراحی کنترل دیجیتال	۳۳
طراحی مشاهده گر	۳۷
فصل نهم: کنترل غیرخطی	۳۹
پاسخ سیستم غیرخطی با کنترل کلاسیک	۳۹
کنترل کننده با $K=0$ مربوط به دینامیک سیستم	۴۰
کنترل کننده با $K=1$ مربوط به دینامیک سیستم	۴۱
کنترل کننده با $K=100$ مربوط به دینامیک سیستم	۴۱

۴۲	 پاسخ سیستم غیرخطی با کنترل کننده مدرن
۴۲	 کنترل کننده طراحی شده با روش مقررات درجه دوم خطی (LQR)
۴۳	 کنترل کننده طراحی شده با روش استقرار قطب مقاوم place
۴۵	 فصل دهم: جمع بندی

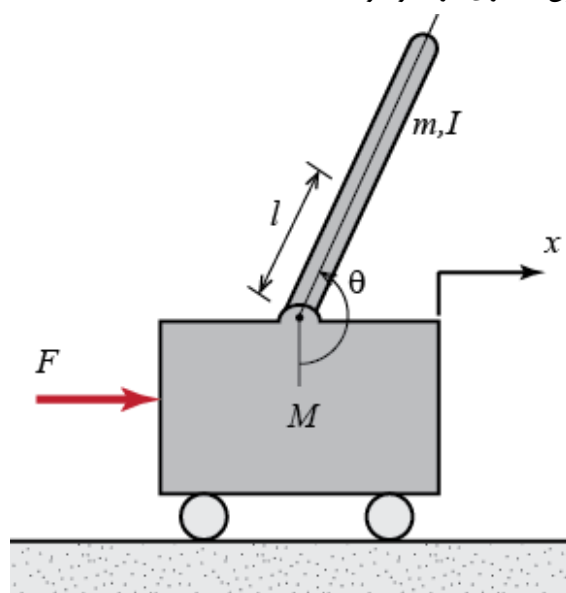
فصل اول: مقدمه

این مسئله از یک آونگ معکوس متصل به یک گاری تشکیل شده است. سیستم آونگ وارونه یک مثال معمول در کتاب های کنترل و منابع تحقیقاتی است. محبوبیت این مسئله از این جا نشأت می شود که این آونگ ناپایدار است و بدون کنترل و حرکت گاری سقوط می کند. علاوه بر این دینامیک این سیستم غیرخطی است.

هدف این سیستم کنترلی، پایدارسازی آونگ معکوس به وسیله اعمال نیرویی به گاری که به آن متصل است. یک مثال در دنیای واقعی که به طور مستقیم به این سیستم آونگ معکوس مربوط می شود مسئله کنترل یک موشک در فاز برخاستن آن است.

مشخصات مسئله

در این حالت آونگ به در فضای ۲ بعدی در نظر گرفته شده است و فقط در صفحه عمودی حرکت می کند. برای این سیستم نیروی کنترلی F که باعث حرکت افقی گاری می شود یک ورودی سیستم است و زاویه آونگ θ و موقعیت افقی گاری x خروجی سیستم می باشند. همچنین فرض شده است که پاندول و چرخ های گاری در طول مسئله کاملاً صلب اند و همچنین هیچ گونه اصطکاکی در مفصل پاندول با گاری در نظر گرفته نشده است.



شکل ۱ - تصویر ساده شده مسئله آونگ معکوس

برای مثال فرض می‌کنیم مسئله‌داری مشخصات زیر است:

نماد	مقدار	توضیحات
M	1 kg	جرم گاری
m	0.1 kg	جرم آونگ
l	1 m	فاصله لولا تا مرکز جرم آونگ
I	0	ممان اینرسی آونگ
g	9.8 m/s^2	شتاب گرانش
F		نیروی وارد شونده به گاری
x		مختصات موقعیت گاری
θ		زاویه آونگ نسبت به خط عمود

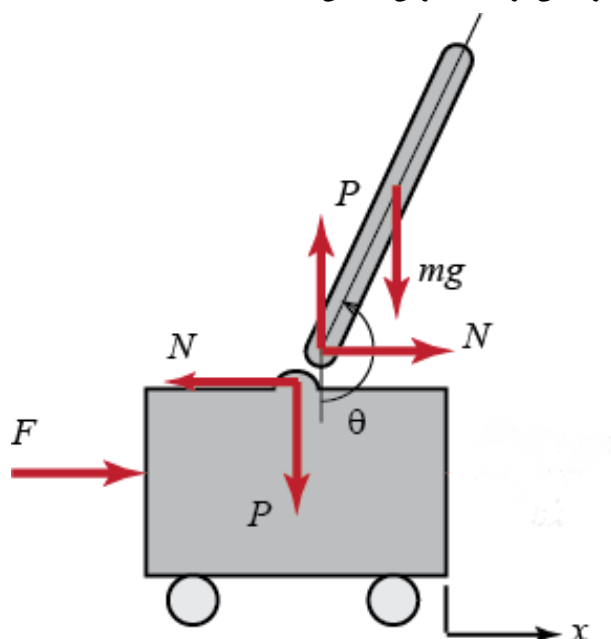
جدول ۱ - مشخصات مسئله

فصل دوم: مدل سازی سیستم

در این فصل قصد داریم معادلات سیستم را به دست آوریم و با خطی سازی آنها تابع تبدیل و معادلات فضای حالت را از آنها استخراج نماییم.

معادلات سیستم system equations

در شکل زیر نمودار جسم آزاد از دو المان آونگ معکوس نمایش داده شده است:



شکل ۲ - نمودار جسم آزاد هر دو المان سیستم آونگ معکوس

پس از جمع نیروهای افقی نمودار جسم آزاد گاری به معادله‌ای به صورت زیر دست می‌یابیم:

$$M\ddot{x} + N = F$$

توجه داشته باشید که همچنین می‌توان نیروهای در جهت عمودی وارد به گاری را می‌توان جمع کرد اما هیچ اطلاعات مفیدی به دست نمی‌آید.

با جمع نیروهای وارد بر آونگ در جهت افقی برای واکنش به نیروی N عبارت زیر به دست می‌آید:

$$N = m\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta$$

اگر این معادله را با معادله قبل ترکیب کنیم و نیروی N را حذف کنیم یکی از معادله حالت حاکم برای این سیستم به صورت زیر به دست می‌آید:

$$(M + m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta = F$$

برای به دست آوردن معادله دوم حالت برای این سیستم می‌توان از مجموع نیروهای عمودی وارد بر آونگ استفاده کرد. حل سیستم در طول محور آونگ محاسبات ریاضی را تا حد زیادی ساده می‌کند و معادله زیر به دست می‌آید:

$$P \sin \theta + N \cos \theta - mg \sin \theta = ml\ddot{\theta} + m\ddot{x} \cos \theta$$

برای حذف نیروهای داخلی N و P از معادله بالا می‌توان معادله مربوط به دوران آونگ را حول مرکز جرم آن نوشت تا معادله زیر به دست آید:

$$-Pl \sin \theta - Nl \cos \theta = I\ddot{\theta}$$

با ترکیب این دو عبارت آخر می‌توان به معادله دوم حالت را به دست آورد و نتیجه به صورت زیر است:

$$(I + ml^2)\ddot{\theta} - mgl \sin \theta = -ml\ddot{x} \cos \theta$$

از آنجایی که تجزیه، تحلیل و تکنیک های طراحی سیستم کنترلی به طور معمول برای سیستم های خطی است این مجموعه از معادلات نیاز دارد که خطی شود. به صورت خاص، معادلات حول موقعیت پایدار و عمودی آونگ یعنی $\theta = \pi$ خطی می‌کنیم و فرض می‌کنیم که سیستم در یک همسایگی کوچکی از موقعیت پایداری باقی بماند. این فرض در شرایطی تقریباً منطقی است که برای سیستم کنترلی ساخته شده زاویه انحراف آونگ بیش از ۲۰ درجه از موقعیت عمودی آونگ نباشد. در صورتی که زاویه انحراف از موقعیت تعادل آونگ را ϕ فرض کنیم؛ زاویه آونگ به صورت $\theta = \phi + \pi$ می‌شود و باز هم فرض کنیم که انحراف ϕ مقداری کوچک است می‌توانیم با تقریب های زیر توابع غیرخطی معادلات سیستم را خطی کنیم.

$$\cos \theta = \cos(\phi + \pi) \approx -1$$

$$\sin \theta = \sin(\phi + \pi) \approx -\phi$$

$$\dot{\theta}^2 = \dot{\phi}^2 \approx 0$$

پس از استفاده از جایگذاری معادلات فوق در معادله حالت سیستم غیرخطی، دو معادله خطی حرکت به دست می‌آید. توجه داشته باشید که u به جای نیروی ورودی F در معادلات جایگزین شده است.

$$(I + ml^2)\ddot{\phi} - mgl\phi = ml\ddot{x}$$

$$(M + m)\ddot{x} - ml\ddot{\phi} = u$$

تابع تبدیل Transfer Function

برای به دست آوردن توابع انتقال از معادله خطی شده سیستم ابتدا باید تبدیل لاپلاس معادلات سیستم را با فرض شرایط اولیه صفر به دست آوریم. نتیجه تبدیل لاپلاس به صورت زیر نشان داده شده است.

$$(I + ml^2)\Phi(s)s^2 - mgl\Phi(s) = mlX(s)s^2$$

$$(M + m)X(s)s^2 - ml\Phi(s)s^2 = U(s)$$

همان طور که می‌دانید تابع تبدیل نشان دهنده رابطه بین ورودی خروجی در یک زمان است. برای پیدا کردن اولین تابع تبدیل برای خروجی $\Phi(s)$ و ورودی $U(s)$ نیاز است که از معادلات فوق $X(s)$ را حذف کنیم. حل معادله اول برای $X(s)$ به این صورت است که:

$$X(s) = \left[\frac{I + ml^2}{ml} - \frac{g}{s^2} \right] \Phi(s)$$

سپس $X(s)$ را در معادله دوم قرار داده تا معادله زیر به دست آید:

$$(M + m) \left[\frac{I + ml^2}{ml} - \frac{g}{s^2} \right] \Phi(s)s^2 - ml\Phi(s)s^2 = U(s)$$

با مرتب کردن معادله فوق تابع تبدیل به صورت زیر به دست می آید:

$$\frac{\Phi(s)}{U(s)} = \frac{\frac{ml}{\left[(M+m)(I+ml^2) - (ml)^2 \right]} s^2}{s^4 - \frac{(M+m)mgl}{\left[(M+m)(I+ml^2) - (ml)^2 \right]} s^2}$$

از تابع تبدیل بالا می توان مشاهده کرد که ۲ قطب و ۲ صفر دقیقاً در مبدأ قرار دارند و این دو می توانند لغو شوند و تابع تبدیل در نهایت به شکل زیر می شود:

$$P_{pend}(s) = \frac{\Phi(s)}{U(s)} = \frac{\frac{ml}{\left[(M+m)(I+ml^2) - (ml)^2 \right]}}{s^2 - \frac{(M+m)mgl}{\left[(M+m)(I+ml^2) - (ml)^2 \right]}} \left[\frac{rad}{N} \right]$$

تابع تبدیل دوم که موقعیت گاری را به عنوان خروجی در نظر گرفته شده را می توان به روش مشابه استخراج کرد و نتیجه به صورت زیر است:

$$P_{cart}(s) = \frac{X(s)}{U(s)} = \frac{\frac{(I+ml^2)}{\left[(M+m)(I+ml^2) - (ml)^2 \right]} s^2 - \frac{mgl}{\left[(M+m)(I+ml^2) - (ml)^2 \right]}}{s^4 - \frac{(M+m)mgl}{\left[(M+m)(I+ml^2) - (ml)^2 \right]} s^2} \left[\frac{m}{N} \right]$$

معادلات فضای حالت State-Space

معادلات خطی حرکت نمایش داده شده در قسمت قبل را اگر به صورت یک سری معادلات دیفرانسیل مرتبه اول مرتب شوند می توان به فرم فضای حالت نشان شود. از آنجایی که معادلات خطی هستند آن ها را می توان به شکل ماتریس های استاندارد زیر نشان داد.

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\Phi} \\ \ddot{\Phi} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{m^2 gl^2}{I(M+m) + Mml^2} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{mgl(M+m)}{I(M+m) + Mml^2} & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \Phi \\ \dot{\Phi} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{I+ml^2}{I(M+m) + Mml^2} \\ 0 \\ \frac{ml}{I(M+m) + Mml^2} \end{bmatrix} u$$

$$y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \Phi \\ \dot{\Phi} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} u$$

ماتریس C به این دلیل ۲ ردیفه است که هر دو متغیر موقعیت گاری و زاویه آونگ به عنوان خروجی سیستم است. به طور خاص، موقعیت گاری المان اول خروجی y و زاویه انحراف آونگ از موضع تعادل المان دوم خروجی y است.

فصل سوم: نمایش در نرم افزار MATLAB

در این فصل قصد داریم تابع تبدیل و معادلات فضای حالت را در نرم افزار MATLAB نمایش دهیم.

تابع تبدیل Transfer Function

می توان توابع انتقال بالا را برای سیستم آونگ معکوس در داخل نرم افزار قدرتمند MATLAB با استفاده از دستورات نمایش داد.

```
M=1;
m=0.1;
I=0;
g=9.8;
l=1;
q=(M+m)*(I+m*l^2)-(m*l)^2;
s=tf('s');
P_cart=((I+m*l^2)/q)*s^2-(m*g*l/q)/(s^4-((M+m)*m*g*l)*s^2/q);
P_pend=(m*l/q)/(s^2-((M+m)*m*g*l)/q);
sys_tf=[P_cart;P_pend];
inputs={'u'};
outputs={'x';'phi'};
set(sys_tf,'InputName',inputs)
set(sys_tf,'OutputName',outputs)
sys_tf
```

دستور ۱ - نمایش تابع تبدیل در نرم افزار MATLAB

اجرای دستور فوق در پنجره فرمان نرم افزار MATLAB خروجی زیر را نمایش می دهد:

```
Transfer function from input "u" to output...
      0.1 s^2 - 0.98
x:  -----
      0.1 s^4 - 1.078 s^2

      1
phi: -----
      s^2 - 10.78
```

دستور ۲ - نتیجه دستورات اراده شده قبلی

معادلات فضای حالت State-Space

همچنین می توانیم سیستم را به وسیله معادلات حالت نمایش دهیم. دستورات زیر معادلات فضای حالت را برای سیستم آونگ معکوس در داخل نرم افزار قدرتمند MATLAB نمایش می دهند.

```
M=1;
m=0.1;
I=0;
g=9.8;
l=1;
p=I*(M+m)+M*m*l^2;
```

```

A=[0,1,0,0;0,0,(m^2*g*l^2)/p,0;0,0,0,1;0,0,m*g*l*(M+m)/p,0];
B=[0;(I+m*l^2)/p;0;m*l/p];
C=[1,0,0,0;0,0,1,0];
D=[0;0];
states={'x','x_dot','phi','phi_dot'};
inputs={'u'};
outputs={'x','phi'};
sys_ss=ss(A,B,C,D,'statename',states,'inputname',inputs,'outputname',outputs)

```

دستور ۳ - نمایش معادلات فضای حالات در نرم افزار MATLAB

اجرای دستور فوق در پنجره فرمان نرم افزار MATLAB خروجی زیر را نمایش می دهد:

```

a =
           x      x_dot      phi  phi_dot
      x          0          1          0          0
    x_dot        0          0      0.98          0
      phi        0          0          0          1
    phi_dot      0          0     10.78          0

b =
           u
      x          0
    x_dot        1
      phi        0
    phi_dot      1

c =
           x      x_dot      phi  phi_dot
      x          1          0          0          0
      phi        0          0          1          0

d =
           u
      x          0
      phi        0

Continuous-time model.

```

دستور ۴ - نتیجه دستورات اراده شده قبلی

مدل فضای حالت فوق را می توان به صورت تابع تبدیل با استفاده از دستور tf همان طور که در زیر نشان داده شده است تبدیل کرد. در مقابل، مدل تابع تبدیل را می توان به فرم فضای حالت با استفاده از دستور ss تبدیل کرد.

```
sys_tf = tf(sys_ss)
```

دستور ۵ - تبدیل مدل فضای حالت به صورت تابع تبدیل

اجرای دستور فوق در پنجره فرمان نرم افزار MATLAB خروجی زیر را نمایش می دهد:

```

Transfer function from input "u" to output...
      s^2 + 2.22e-015 s - 9.8
x:  -----
      s^4 - 4.441e-016 s^3 - 10.78 s^2

      1
phi: -----
      s^2 - 4.441e-016 s - 10.78

```

دستور ۶ - نتیجه دستورات اراده شده قبلی

با بررسی نتیجه فوق، توجه داشته باشید عباراتی با ضرایب بسیار کوچک در نتیجه وجود دارد. این عبارات در واقع صفر هستند و نشان‌دهنده جمع شدن خطای گرد کردن الگوریتم نرم‌افزار MATLAB می‌باشند. اگر این عبارات را صفر در نظر بگیریم به همان تابع تبدیل به‌دست آمده در قسمت های قبل می‌رسیم و کاملاً مطابقت دارد.

فصل چهارم: تجزیه و تحلیل سیستم

از مسئله آونگ معکوس توابع تبدیل حلقه باز زیر مشتق شده‌اند.

$$P_{pend}(s) = \frac{\Phi(s)}{U(s)} = \frac{\frac{ml}{s^2 - \frac{(M+m)(I+ml^2)-(ml)^2}{(M+m)mgl}}}{\left[\frac{rad}{N} \right]}$$

$$P_{cart}(s) = \frac{X(s)}{U(s)} = \frac{\frac{(I+ml^2)}{s^4 - \frac{(M+m)mgl}{(M+m)(I+ml^2)-(ml)^2}} - \frac{mgl}{s^2 - \frac{(M+m)(I+ml^2)-(ml)^2}{(M+m)mgl}}}{\left[\frac{m}{N} \right]}$$

از فصول قبل به یاد دارید که توابع تبدیل فوق زمانی معتبر هستند که مقدار زاویه انحراف آونگ از حالت عمودی ϕ کمتر از ۲۰ درجه باشد و همچنین زاویه مطلق آونگ θ برابر $\pi + \phi$ است. با توجه به پاسخ سیستم آونگ معکوس به یک ضربه‌ای که به گاری اعمال می‌شود و برابر 1 N.sec است، الزامات طراحی برای آونگ عبارت‌اند از:

- زمان نشست برای زاویه θ کمتر از 5 sec باشد.
- زاویه انحراف آونگ ϕ هیچ‌گاه بیشتر از 0.5 rad نباشد.
- علاوه بر این، شرایط مورد نیاز برای ورودی پله 0.2 m به موقعیت گاری به صورت زیر است:
- زمان نشست برای θ و x کمتر از 5 sec باشد.
- زمان صعود برای x کمتر از 0.5 sec باشد.
- زاویه انحراف آونگ از 20 deg یا 0.35 rad بیشتر نباشد.

پاسخ ضربه سیستم حلقه باز

ابتدا به پاسخ حلقه باز سیستم آونگ معکوس توجه می‌کنیم. یک m-file جدید در نرم‌افزار MATLAB ایجاد کرده و از دستور زیر استفاده می‌کنیم تا مدل سیستم را شبیه‌سازی کنیم.

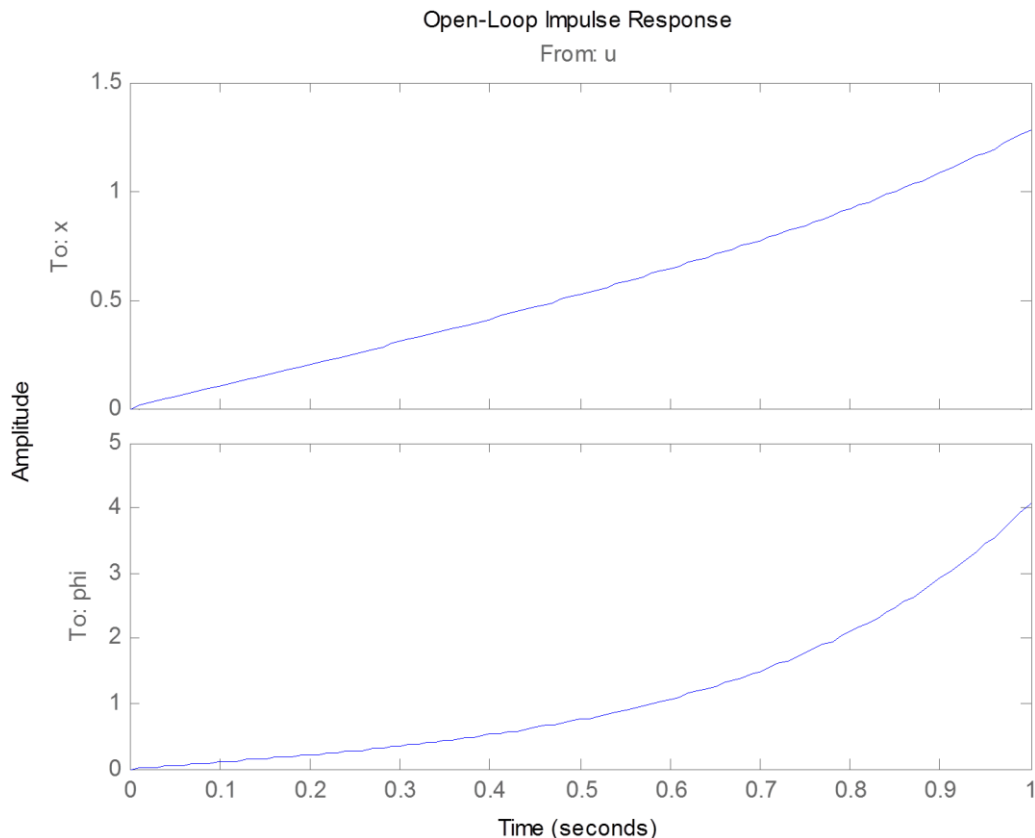
```
M=1;
m=0.1;
I=0;
g=9.8;
l=1;
q=(M+m)*(I+m*l^2)-(m*l)^2;
s=tf('s');
P_cart=((I+m*l^2)/q)*s^2-(m*g*l/q)/(s^4-((M+m)*m*g*l)*s^2/q);
P_pend=(m*l/q)/(s^2-((M+m)*m*g*l)/q);
sys_tf=[P_cart;P_pend];
```

```
inputs={'u'};
outputs={'x';'phi'};
set(sys_tf,'InputName',inputs)
set(sys_tf,'OutputName',outputs)
```

حال می‌توانیم پاسخ سیستم حلقه باز به ورودی ضربه را بررسی کنیم. به طور خاص، به بررسی چگونگی پاسخ به یک نیروی ضربه به گاری را با دستور زیر در MATLAB بررسی می‌کنیم.

```
t=0:0.01:1;
impulse(sys_tf,t);
title('Open-Loop Impulse Response')
```

با اضافه کردن دستور فوق به دستورات قبلی و اجرا کردن آن‌ها در نرم‌افزار به نمودار زیر دست می‌یابیم.



با بررسی این دو نمودار می‌توان به این نتیجه رسید که پاسخ سیستم به طور کامل غیرقابل قبول است. در واقع، این سیستم به صورت حلقه باز ناپایدار است. با این وجود افزایش زاویه آونگ به اندازه 4 rad است که مدل‌سازی فقط برای زاویه‌های کوچک ϕ انجام شده و پاسخ برای مقادیر زیاد آن دقیق نیست. همچنین از نمودار مربوط به موقعیت گاری می‌توان به این نتیجه رسید که موقعیت آن در اثر ضربه بدون تغییر به بینهایت می‌رود.

قطب یک سیستم می‌تواند در مورد پاسخ زمانی یک سیستم اطلاعات خوبی دهد. از آنجاکه این سیستم دو خروجی و یک ورودی دارد با ۲ تابع تبدیل توضیح داده می‌شود. به طور کلی، تمام توابع تبدیل در سیستم‌های چند ورودی و چند خروجی MIMO دارای قطب یکسان‌اند (صفرهای متفاوت هستند) مگر اینکه صفری باعث لغو قطب گردیده باشد. به طور خاص با استفاده از دستور zpkdata نرم‌افزار MATLAB به استخراج قطب و صفر سیستم می‌پردازیم. پارامتر 'v' نشان داده شده در دستور زیر مقادیر قطب و صفر را به صورت نمودار نشان می‌دهد و نه به صورت آرایه‌ای.

```
[zeros poles] = zpkdata(P_pend,'v')
```

که نتیجه دستور فوق به صورت زیر است:

```
zeros =  
  
3.1305
```

```

-3.1305

poles =

    0
    0
 3.2833
-3.2833

```

و همین ترتیب، صفر و قطب سیستم برای تابع تبدیلی که خروجی آن موقعیت گاری است از دستور زیر استفاده می‌کنیم:

```
[zeros poles] = zpkmdata(P_cart, 'v')
```

که نتیجه دستور فوق به صورت زیر است:

```

zeros =

    3.1305
   -3.1305

poles =

    0
    0
 3.2833
-3.2833

```

همان طور که پیش‌بینی شده بود قطب هر دو تابع تبدیل یکسان است. از قطب 3.2833 به دلیل داشتن مقدار حقیقی مثبت، متوجه می‌شویم سیستم ناپایدار است. به عبارت دیگر، این قطب در سمت راست صفحه مختلط s -plane قرار دارد. این موضوع دقیقاً روند ناپایداری را در نمودارهای فوق توضیح می‌دهد.

پاسخ پله سیستم حلقه باز

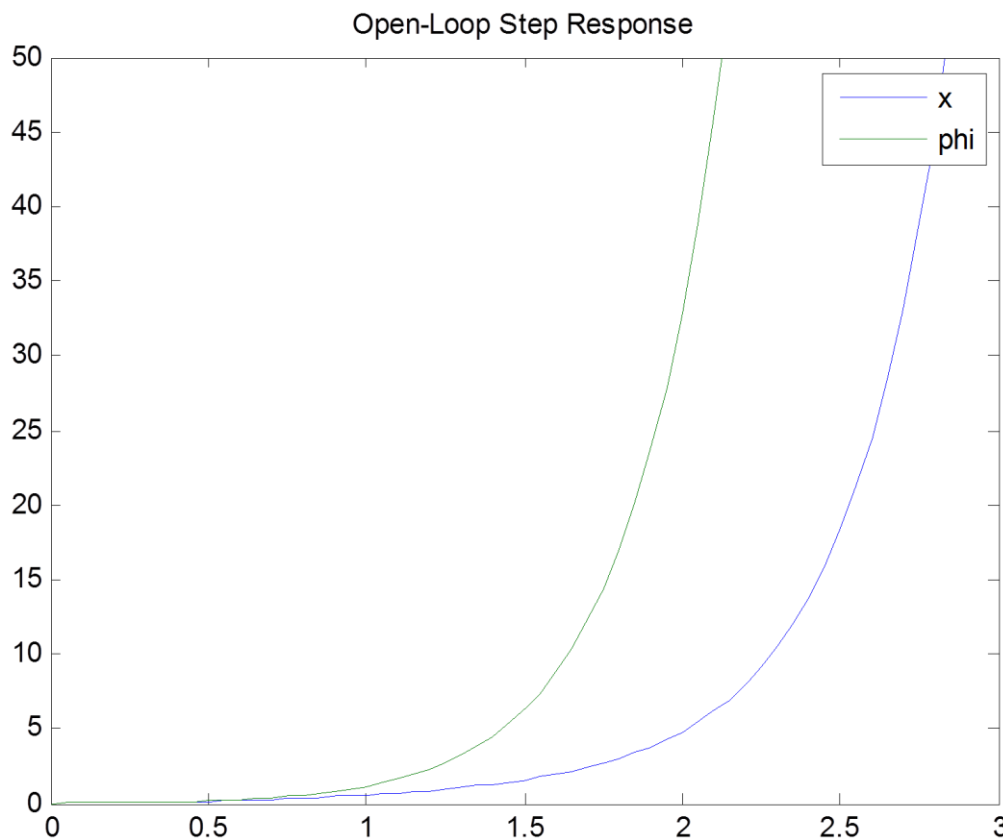
از آنجاکه سیستم دارای یک قطب با مقدار حقیقی مثبت است پاسخ به ورودی پله آن نیز ناپایدار است. به وسیله دستور `lsim` می‌توان پاسخ سیستم LTI را به ورودی دلخواه شبیه‌سازی کرد. در این مورد، ورودی پله را به اندازه 1 Newton استفاده شده است.

```

t = 0:0.05:10;
u = ones(size(t));
[y,t] = lsim(sys_tf,u,t);
plot(t,y)
title('Open-Loop Step Response')
axis([0 3 0 50])
legend('x', 'phi')

```

اضافه کردن دستور فوق به `m-file` و اجزای آن نمودار زیر را نتیجه می‌دهد.



همچنین می‌توانید برخی از ویژگی‌های پاسخ را با استفاده از دستور `lsiminfo` تشخیص داد.

```
step_info = lsiminfo(y,t);
cart_info = step_info(1)
pend_info = step_info(2)
```

که نتیجه دستور فوق به صورت زیر است.

```
cart_info =

    SettlingTime: 9.9934
           Min: 0
    MinTime: 0
           Max: 7.6579e+011
    MaxTime: 10

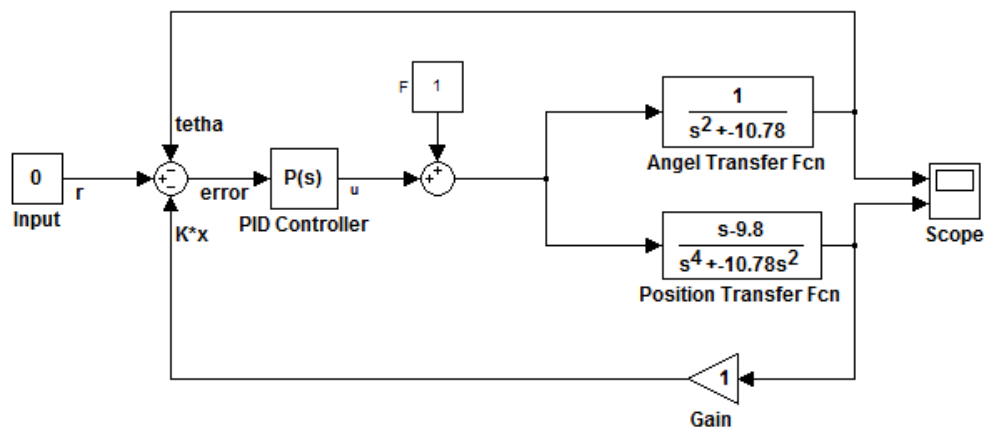
pend_info =

    SettlingTime: 9.9934
           Min: 0
    MinTime: 0
           Max: 8.4237e+012
    MaxTime: 10
```

نتایج فوق تأیید کننده آن است که سیستم به ورودی پله ناپایدار است. از تجزیه و تحلیل‌های فوق این گونه استنباط می‌شود که به کنترلی برای بهبود پاسخ سیستم نیاز است.

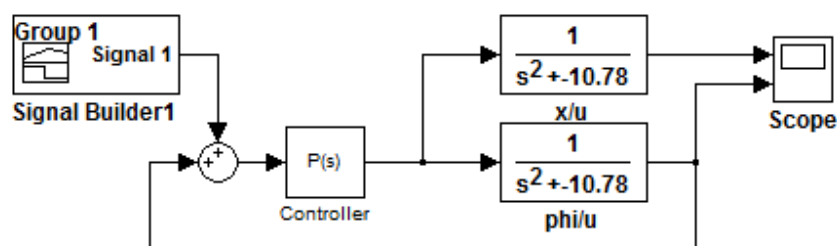
فصل پنجم: طراحی کنترل کننده

در سؤال مطرح شده از طرف دکتر محمدجعفر صدیق خواسته شده کنترلی برای سیستم زیر طراحی شود تا سیستم پایدار شود و پاسخ مطلوبی داشته باشد. علاوه بر این جمع کردن سیگنال های ϕ و x به کلی غلط است زیرا این دو سیگنال از جنس های متفاوت زاویه و موقعیت بر حسب رادیان و متر هستند علاوه بر این سیستم چند ورودی و چند خروجی است و امکان طراحی کنترل گر بر مبنای کنترل کلاسیک نیست زیرا زمانی که شروع به طراحی می کنیم نمی دانیم که کدام خروجی را برای محاسبه تابع تبدیل حلقه بسته ملاک قرار دهیم با این وجود برای این سیستم می توان یک کنترل PID طراحی می کنیم. البته در این بخش از جبرانساز استفاده نشده و فقط با روش صفر-قطب حذف شونده سیستم را پایدار کرده و نتیجه را بررسی می کنیم.



طراحی کنترل کننده با $K=0$ مربوط به دینامیک سیستم

با $k=0$ سیستم به صورت زیر ساده می شود.



پس تابع تبدیل ورودی نیرو به خروجی زاویه آونگ با استفاده از دستور زیر به دست می آید.

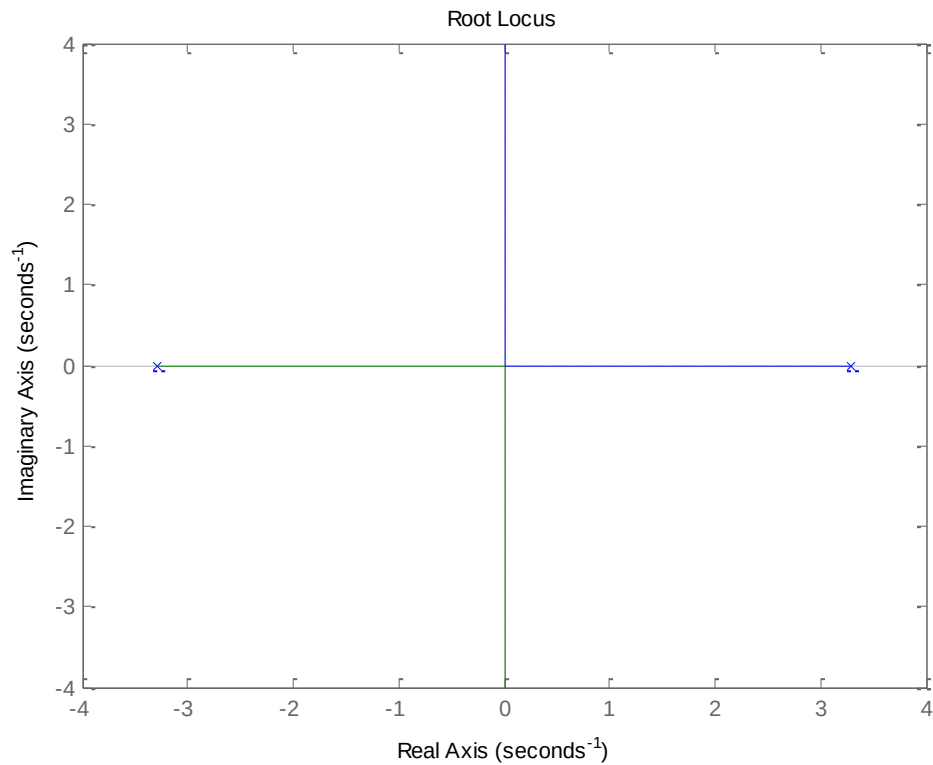
```
sys=tf([1],[1 0 -10.78]);
inputs={'u'};
outputs={'K*x+phi'};
set(sys,'InputName',inputs)
set(sys,'OutputName',outputs)
sys
rlocus(sys)
```

که نتیجه دستور فوق به صورت زیر است.

Transfer function from input "u" to output "K*x+phi":

$$\frac{1}{s^2 - 10.78}$$

و نمودار مکان هندسی ریشه‌ها به صورت زیر است.



و با استفاده از دستور زیر مقادیر قطب و صفر به دست می‌آید.

```
[zeros poles]=zpkdata(sys, 'v')
```

که نتیجه دستور فوق به صورت زیر است.

```
zeros =  
  
Empty matrix: 0-by-1  
  
poles =  
  
    3.2833  
   -3.2833
```

همان طور که مشاهده می‌کنید ۱ قطب سمت راست محور اعداد حقیقی قرار دارد و سیستم را ناپایدار می‌سازد.

و در کنترلر یک قطب در ۴- قرار می‌دهیم و یک صفر در ۳.۲۸۳۳ قرار می‌دهیم.

```
sysc=zpk([(10.78)^0.5], [-4], 1);  
tf(sysc)
```

که نتیجه دستور فوق به صورت زیر است.

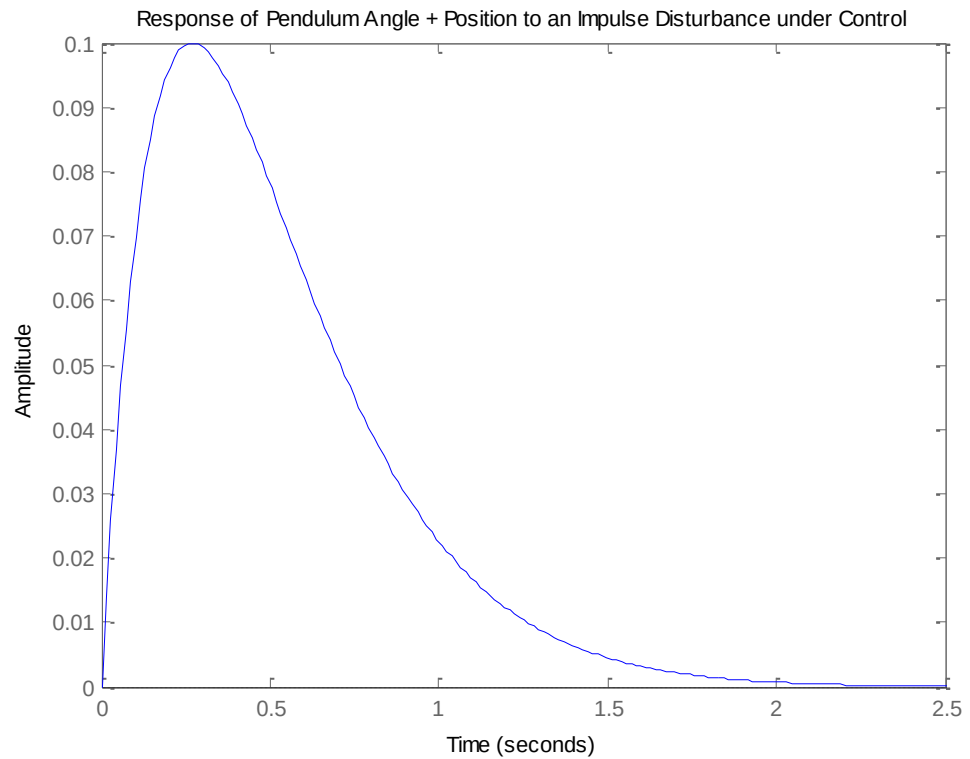
```
Transfer function:  
s - 3.283  
-----  
s + 4
```

و به صورت زیر کارایی سیستم را بررسی می‌کنیم.

```
openloop=zpk([], [-3.2833, -4], 1);  
rlocus(openloop);  
syst=tf(openloop)
```

```
T=syst/(1+syst);
impulse(T)
title('Response of Pendulum Angle + Position to an Impulse Disturbance
under Control');
```

و نتیجه به صورت زیر است.



البته می توان با افزایش ضریب کنترلر آن را بهبود بخشید.

طراحی کنترل کننده با $K=1$ مربوط به دینامیک سیستم

تابع تبدیل بخش های این آونگ معکوس در فصل قبل به دست آمده و به صورت زیر است.

$$P_{pend}(s) = \frac{\Phi(s)}{U(s)} = \frac{\frac{ml}{s^2 - \frac{[(M+m)(I+ml^2) - (ml)^2]}{(M+m)mgl}}}{\left[\frac{rad}{N} \right]}$$

$$P_{cart}(s) = \frac{X(s)}{U(s)} = \frac{\frac{(I+ml^2)}{s^4 - \frac{[(M+m)(I+ml^2) - (ml)^2]}{(M+m)mgl}} s^2 - \frac{mgl}{[(M+m)(I+ml^2) - (ml)^2]}}{\left[\frac{m}{N} \right]}$$

لذا تابع تبدیل موقعیت گاری به زاویه

$$X(s) = \left[\frac{I+ml^2}{ml} - \frac{g}{s^2} \right] \Phi(s) \rightarrow \frac{\Phi(s)}{X(s)} = \left[\frac{I+ml^2}{ml} - \frac{g}{s^2} \right] = \frac{s^2 - 9.8}{s^2}$$

آونگ به صورت زیر است.

که با استفاده از نرم افزار MATLAB به صورت زیر نمایش داده می شود.

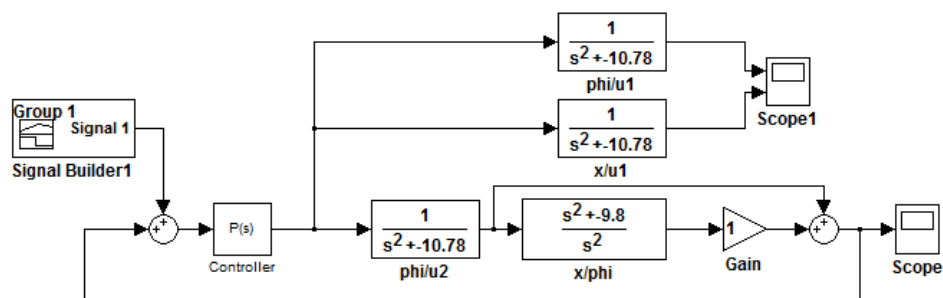
```
sys=tf([1 0 0],[1 0 -9.8]);
inputs={'phi'};
outputs={'x'};
set(sys,'InputName',inputs)
set(sys,'OutputName',outputs)
sys
```

نتیجه دستور فوق به صورت زیر است.

Transfer function from input "phi" to output "x":

$$\frac{s^2}{s^2 - 9.8}$$

پس شکل سیستم کنترلی به صورت زیر بازسازی می شود



پس تابع تبدیل ورودی نیرو به خروجی زاویه آونگ به علاوه موقعیت گاری با استفاده از دستور زیر به دست می آید.

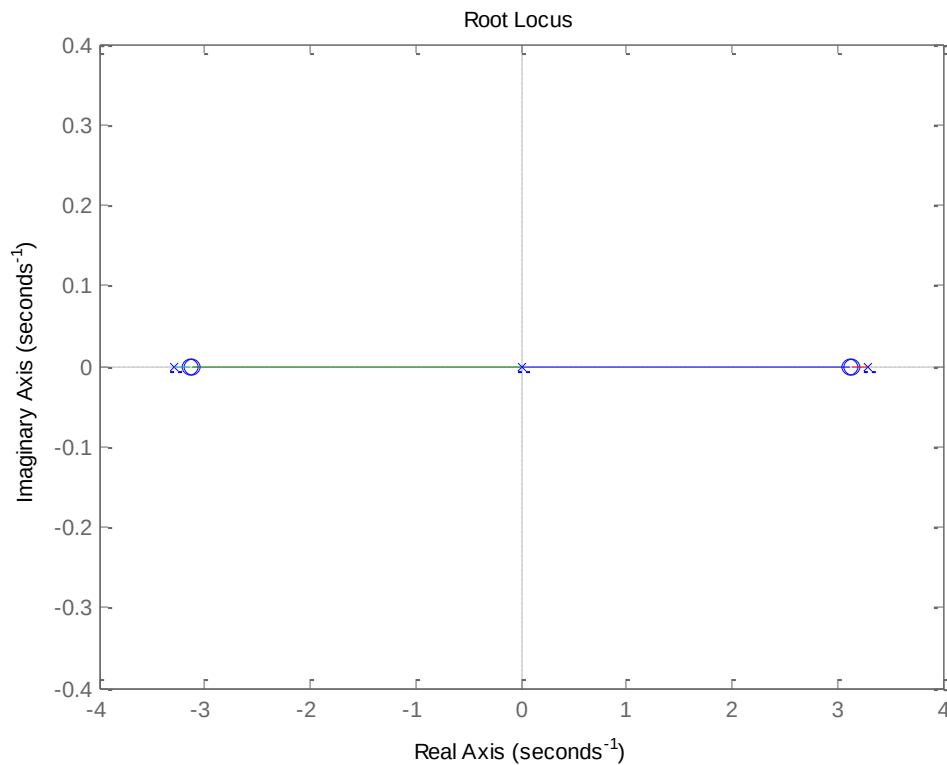
```
K=1;
sys1=tf([1 0 -9.8],[1 0 0]);
sys2=tf([1],[1 0 -10.78]);
sys=K*sys1*sys2+sys1;
inputs={'u'};
outputs={'K*x+phi'};
set(sys,'InputName',inputs)
set(sys,'OutputName',outputs)
sys
rlocus(sys)
```

که نتیجه دستور فوق به صورت زیر است.

Transfer function from input "u" to output "K*x+phi":

$$\frac{s^6 - 19.58 s^4 + 95.84 s^2}{s^6 - 10.78 s^4}$$

و نمودار مکان هندسی ریشه ها به صورت زیر است.



و با استفاده از دستور زیر مقادیر قطب و صفر به دست می آید.

```
[zeros poles]=zpkdata(sys,'v')
```

که نتیجه دستور فوق به صورت زیر است.

```
zeros =
    0
    0
   -3.1305
   -3.1273
    3.1305
    3.1273

poles =
    0
    0
    0
    0
    3.2833
   -3.2833
```

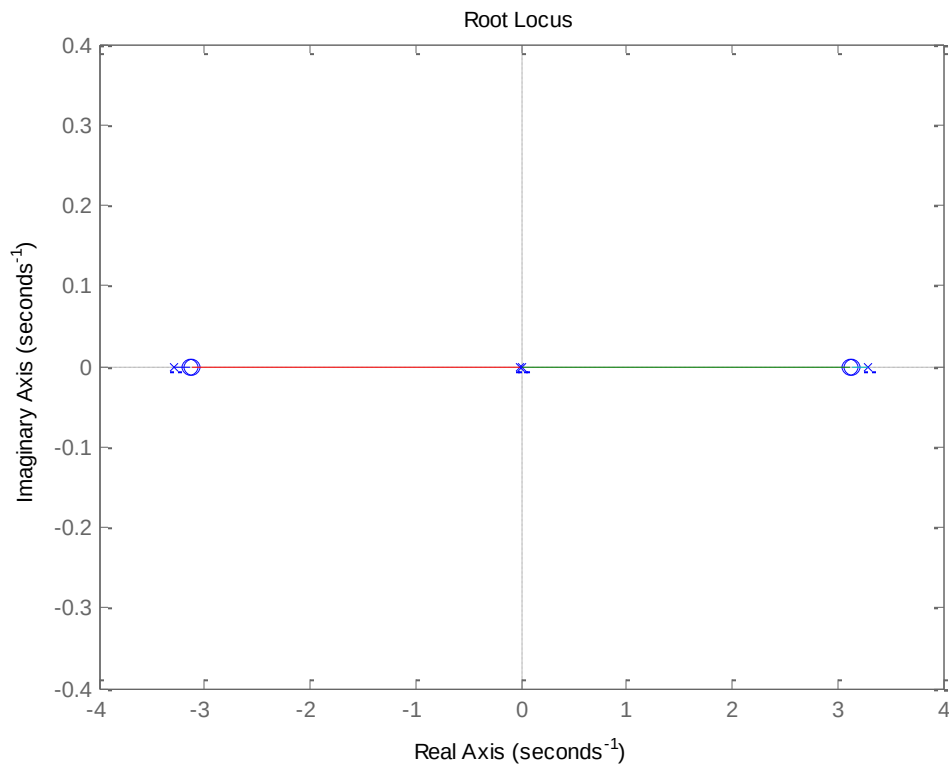
همان طور که مشاهده می کنید ۲ صفر و ۱ قطب سمت راست محور اعداد حقیقی قرار دارد و سیستم را ناپایدار می سازد و به دلیل وجود صفر امکان طراحی کنترلر نیست. و سیستم ساده شده به صورت زیر است.

```
sys=zpk([-3.1305,-3.1273,3.1305,3.1273],[0,0,3.2833,-3.2833],1);
tf(sys)
rlocus(sys)
```

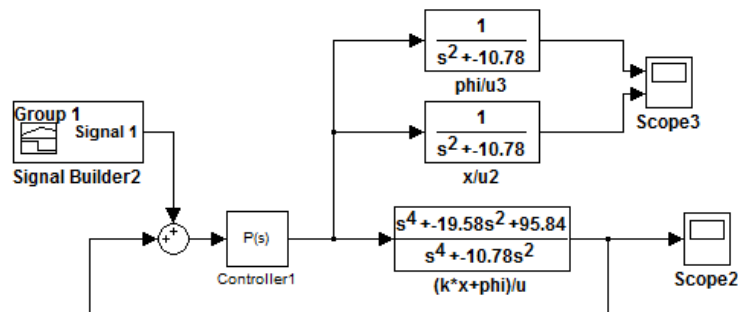
که نتیجه آن به صورت زیر است.

```
Transfer function:
s^4 - 19.58 s^2 + 95.84
-----
s^4 - 10.78 s^2
```

و نمودار مکان هندسی ریشه ها به صورت زیر است.



همان طور که مشاهده می کنید باز هم یک صفر سمت راست دارد و سیستم کنترل ناپذیر است ولی می توان در کنترلر یک قطب در آن نقطه قرار داد تا اثر آن را خنثی کند. به هر حال سیستم ساده شده به صورت زیر است.



و در کنترلر یک قطب در 3.1273 قرار می دهیم و یک صفر در 3.2833 قرار می دهیم.

```
sysc=zpk([3.2833],[ 3.1305,3.1273],0.05);
tf(sysc)
```

که نتیجه دستور فوق به صورت زیر است.

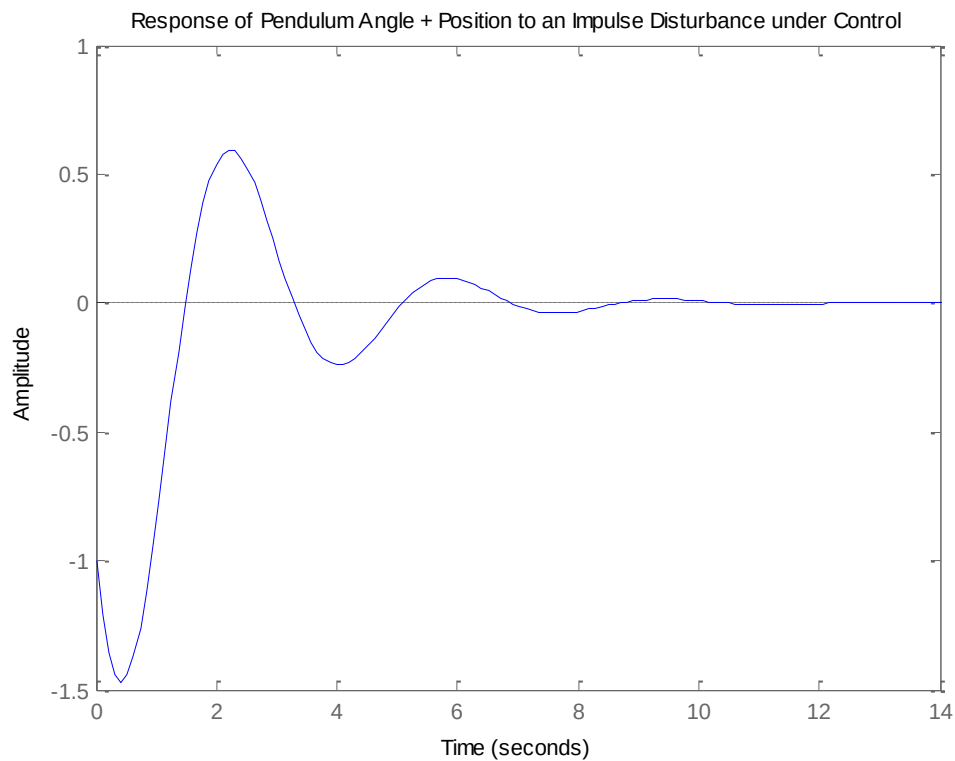
```
Transfer function:
  0.05 s - 0.1642
-----
s^2 - 6.258 s + 9.79
```

و به صورت زیر کارایی سیستم را بررسی می کنیم.

```
sys=zpk([0,0,-3.1273,3.1273],[-3.2833,-3.1305,3.2833],1);
sysc=zpk([3.2833],[3.1305],1);
openloop=zpk([0,0,-3.1273],[-3.2833,-3.1305],1);
rlocus(openloop);
syst=tf(openloop)
T=syst/(1+syst);
impulse(T)
```

```
title('Response of Pendulum Angle + Position to an Impulse Disturbance under Control');
```

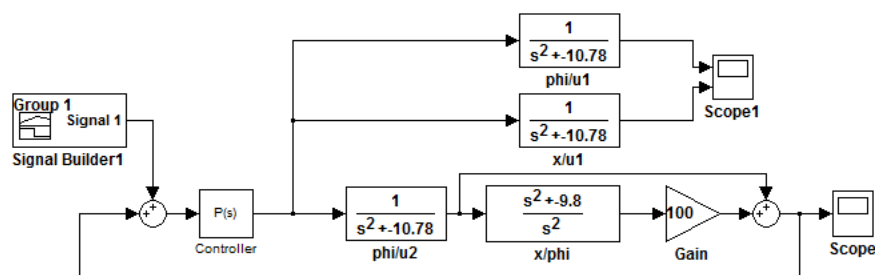
و نتیجه به صورت زیر است.



البته می توان با افزایش ضریب کنترلر آن را بهبود بخشید.

طراحی کنترل کننده با $K=100$ مربوط به دینامیک سیستم

شکل سیستم کنترلی به صورت زیر بازسازی می شود.



پس تابع تبدیل ورودی نیرو به خروجی زاویه آونگ به علاوه موقعیت گاری با استفاده از دستور زیر به دست می آید.

```
K=100;
sys1=tf([1 0 -9.8],[1 0 0]);
sys2=tf([1],[1 0 -10.78]);
sys=K*sys1*sys2+sys1;
inputs={'u'};
outputs={'K*x+phi'};
set(sys,'InputName',inputs)
set(sys,'OutputName',outputs)
sys
```

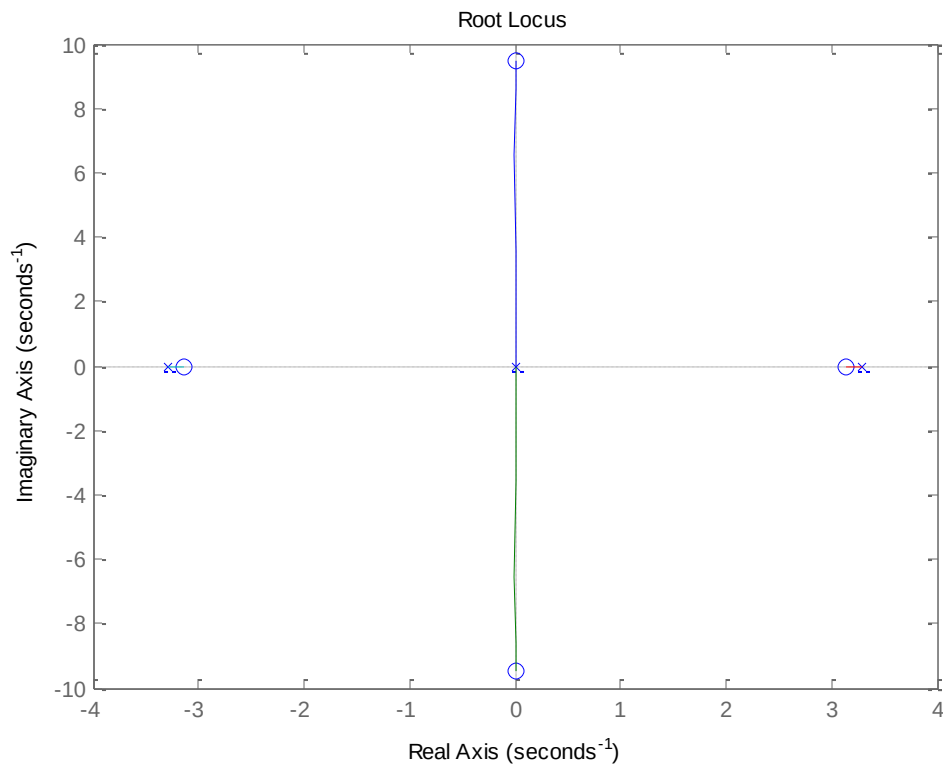
```
rlocus(sys)
```

که نتیجه دستور فوق به صورت زیر است.

Transfer function from input "u" to output "K*x+phi":

$$\frac{s^6 + 79.42 s^4 - 874.4 s^2}{s^6 - 10.78 s^4}$$

و نمودار مکان هندسی ریشه‌ها به صورت زیر است.



و با استفاده از دستور زیر مقادیر قطب و صفر به دست می‌آید.

```
[zeros poles]=zpkdata(sys,'v')
```

که نتیجه دستور فوق به صورت زیر است.

```
zeros =  
  
      0  
      0  
  0.0000 + 9.4456i  
  0.0000 - 9.4456i  
 -3.1305  
  3.1305  
  
poles =  
  
      0  
      0  
      0  
      0  
  3.2833  
 -3.2833
```

```
sys=zpk([9.4456i,-9.4456i,-3.1305,3.1305],[0,0,3.2833,-3.2833],1);  
tf(sys)  
rlocus(sys)
```

Transfer function:

$$\frac{s^4 + 79.42 s^2 - 874.4}{s^4 - 10.78 s^2}$$

Root Locus

Imaginary Axis (seconds⁻¹)

Real Axis (seconds⁻¹)

```
sysc=zpk([3.2833],[3.1305],1);  
tf(sysc)
```

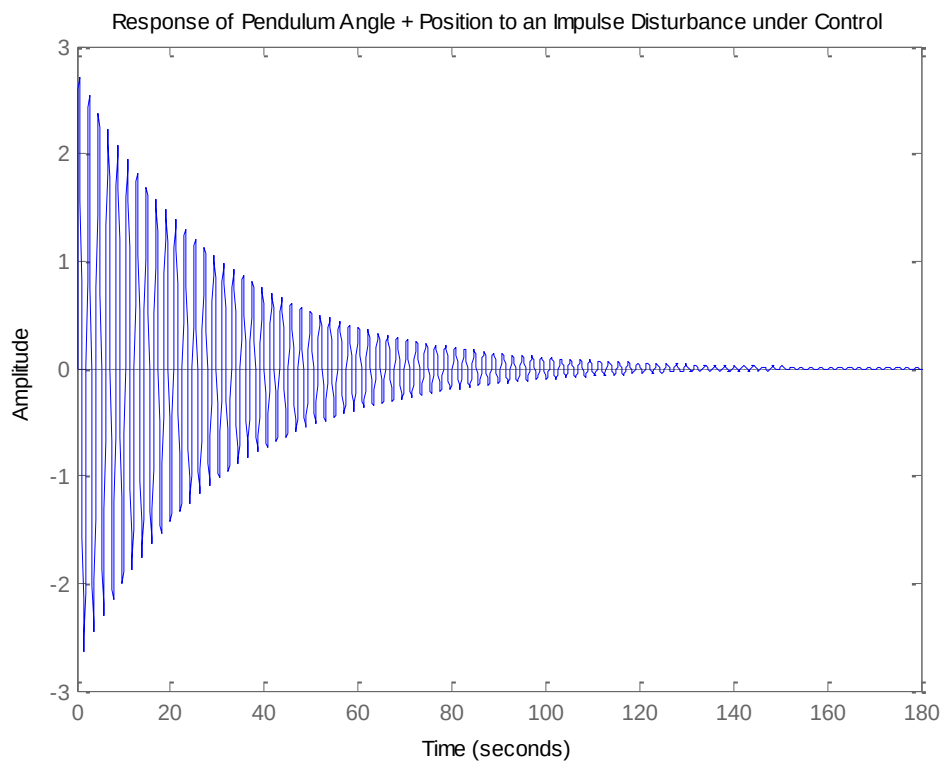

که نتیجه دستور فوق به صورت زیر است.

```
Transfer function:
s - 3.283
-----
s - 3.131
```

و به صورت زیر کارایی سیستم را بررسی می کنیم.

```
openloop=zpk([9.4456i,-9.4456i,-3.1305],[0,0,-3.2833],0.12);
rlocus(openloop);
syst=tf(openloop)
T=syst/(1+syst);
impulse(T)
title('Response of Pendulum Angle + Position to an Impulse Disturbance
under Control');
```

و نتیجه به صورت زیر است.



البته می توان با تغییر کنترلر آن را بهبود بخشید و یا حتی از جبران ساز های پیشرو و پسرو استفاده کنیم. به علت سنگین بودن پروژه از بهبود این سیستم صرف نظر می کنیم.

فصل ششم: روش فضای حالت برای طراحی کنترل

از مسئله اصلی، معادلات دینامیک سیستم آونگ معکوس به فرم فضای حالت به صورت زیر شرح داده شده است.

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\Phi} \\ \ddot{\Phi} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0.98 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 10.78 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \Phi \\ \dot{\Phi} \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix} u$$

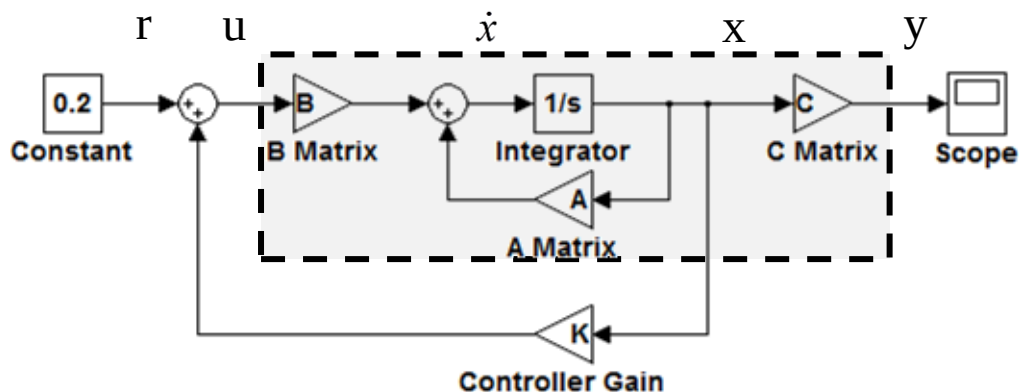
$$y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \Phi \\ \dot{\Phi} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} u$$

معیار های طراحی برای انتقال موقعیت گاری x به اندازه 0.2 m به صورت تابع پله به صورت زیر است:

- زمان نشست برای x و θ کمتر از 5 sec باشد.
- زمان صعود برای x کمتر از 0.5 sec باشد.
- زاویه انحراف آونگ ϕ نباید بیشتر از 20 deg یا 0.35 rad از حالت عمودی آن باشد.
- خطای حالت ماندگار کمتر از 2% برای x و θ باشد.

در این بخش قصد داریم تا با طراحی کنترلر مناسب آونگ را به صورت عمودی نگاه داریم در صورتی که موقعیت گاری به اندازه 0.2 m به سمت راست حرکت کرده است. رویکرد طراحی در فضای حالت به این صورت است که به خوبی برای سیستم های چند خروجی کنترلر طراحی می کند.

این مسئله را می توان با استفاده از بازخورد حالت کامل حل کرد. طرح کلی این نوع سیستم کنترل در شکل زیر نشان داده شده است و K ماتریس ضرایب کنترلی است. توجه داشته باشید که تمام متغیر های حالت را به جای استفاده از خروجی سیستم بازخورد دادیم.



قطب های حلقه باز Open-loop poles

در این مسئله r نشان دهنده ورودی پله به موقعیت گاری است. متغیر حالت تشکیل شده از موقعیت و سرعت گاری و زاویه و سرعت زاویه آونگ است. خروجی y نشان دهنده موقعیت گاری و زاویه آونگ است. قصد داریم یک کنترلی طراحی کنیم که در صورتی که یک ورودی پله به سیستم وارد شد و آونگ تغییر زاویه داد در نهایت کنترلر آن را به حالت عمودی برگرداند و گاری به موقعیت جدید برود.

اولین گام برای طراحی کنترلر از بازخورد حالت کامل تعیین قطب های حلقه باز سیستم است. با وارد کردن دستور زیر در یک m-file می توان قطب های حلقه باز را از مقادیر ویژه ماتریس A به دست آورد.

```

M=1;
m=0.1;
I=0;
g=9.8;
l=1;
p=I*(M+m)+M*m*l^2;
A=[0,1,0,0;0,0,(m^2*g*l^2)/p,0;0,0,0,1;0,0,m*g*l*(M+m)/p,0];
B=[0;(I+m*l^2)/p;0;m*l/p];
C=[1,0,0,0;0,0,1,0];
D=[0;0];
states={'x','x_dot','phi','phi_dot'};
inputs={'u'};
outputs={'x';'phi'};
sys_ss=ss(A,B,C,D,'statename',states,'inputname',inputs,'outputname',outputs);
poles = eig(A)

```

که نتیجه دستور فوق به صورت زیر می شود.

```

poles =

      0
      0
  3.2833
 -3.2833

```

همان طور که ملاحظه می کنید قطب ۳.۲۸۳۳ در نیمه سمت راست صفحه قرار دارد و نشان دهنده این است که سیستم حلقه باز ناپایدار است.

روش مقررات درجه دوم خطی (LQR)

مرحله بعدی در روند طراحی پیدا کردن بردار ضرایب کنترلی K با فرض اندازه گیری کامل متغیرهای حالت است. پیدا کردن بردار ضرایب کنترلی به روش های مختلفی قابل انجام است. در صورتی که محل قطب حلقه باز مطلوب مشخص باشد می توان از دستورات place و یا acker در نرم افزار MATLAB استفاده کرد. روشی دیگر این است که از دستور lqr استفاده شود که ضرایب کنترلی بهینه شده مطلوبی را با توجه به سیستم خطی، تابع درجه دو، و مراجع برابر صفر ارائه می دهد. قبل از طراحی کنترلر به بررسی کنترل پذیری سیستم پرداخته می شود. وجود کنترل پذیری نشان دهنده آن است که متغیرهای حالت را در هر مکانی و در هر زمانی که می خواهیم ببریم. برای سیستم کنترل پذیر کامل باید ماتریس کنترل پذیری دارای رتبه n باشد به این معنی که دارای تعداد ردیف (یا ستون) مستقل باشد. ماتریس کنترل پذیری به صورت زیر به دست می آید. که تعداد n مربوط به تعداد متغیرهای حالت است.

$$C = [A \mid AB \mid A^2B \mid \dots \mid A^{n-1}B]$$

از آن جایی که ماتریس کنترل پذیری برای سیستم ما 4×4 است باید رتبه ماتریس ۴ باشد. با استفاده از دستور ctrb نرم افزار MATLAB می توان ماتریس کنترل پذیری سیستم را استخراج نمود و با استفاده از دستور rank می توان مرتبه ماتریس را بررسی کرد.

```

co=ctrb(sys_ss)
controllability=rank(co)

```

با اضافه کردن دستور فوق به دستورات قبلی در پنجره فرمان نرم افزار MATLAB خروجی زیر تولید می شود.

```

co =

      0      1.0000      0      0.9800
  1.0000      0      0.9800      0

```

0	1.0000	0	10.7800
1.0000	0	10.7800	0

controllability =

4

بنابراین متوجه شدیم که سیستم کنترل پذیر است و در نتیجه ما باید قادر به طراحی یک کنترلر برای دستیابی به سیستم پایدار هستیم. به طور خاص با استفاده از روش LQR برای تعیین ماتریس ضرایب کنترلی K استفاده می کنیم. تابع `lqr` نرم افزار MATLAB به کاربر اجازه می دهد که دو پارامتر Q و R را انتخاب کنیم که متعادل کننده اهمیت کنترل u و خطا e (انحراف از ۰) است که به ترتیب در تابع بهینه سازی است. ساده ترین حالت آن است که فرض کنیم $R=1$ و $Q=C'C$ باشد که نشان دهنده این است که کنترل زاویه آونگ و موقعیت گاری به یک میزان اهمیت دارد. برای مشاهده ساختار Q می توان از دستور زیر در نرم افزار MATLAB استفاده کرد.

$Q=C'*C$

که نتیجه استفاده از دستور فوق به صورت زیر است.

$Q =$

1	0	0	0
0	0	0	0
0	0	1	0
0	0	0	0

المان ردیف اول و ستون اول نشان دهنده وزن کنترل موقعیت گاری و المان ردیف سوم و ستون سوم نشان دهنده وزن کنترل زاویه آونگ در طراحی کنترلر بهینه است. و در نهایت ماتریس R بیان کننده اهمیت ارزش نسبی R و Q نسبت به هم است و ارزش مطلق آنها نیست. حال با استفاده از دو ماتریس فوق به پیدا کردن تعیین ماتریس ضرایب کنترلی K می پردازیم.

$Q=C'*C;$
 $R=1;$
 $K=lqr(A,B,Q,R)$

با اضافه کردن دستور فوق به دستورات قبلی در پنجره فرمان نرم افزار MATLAB می توان را به صورت زیر به دست آورد

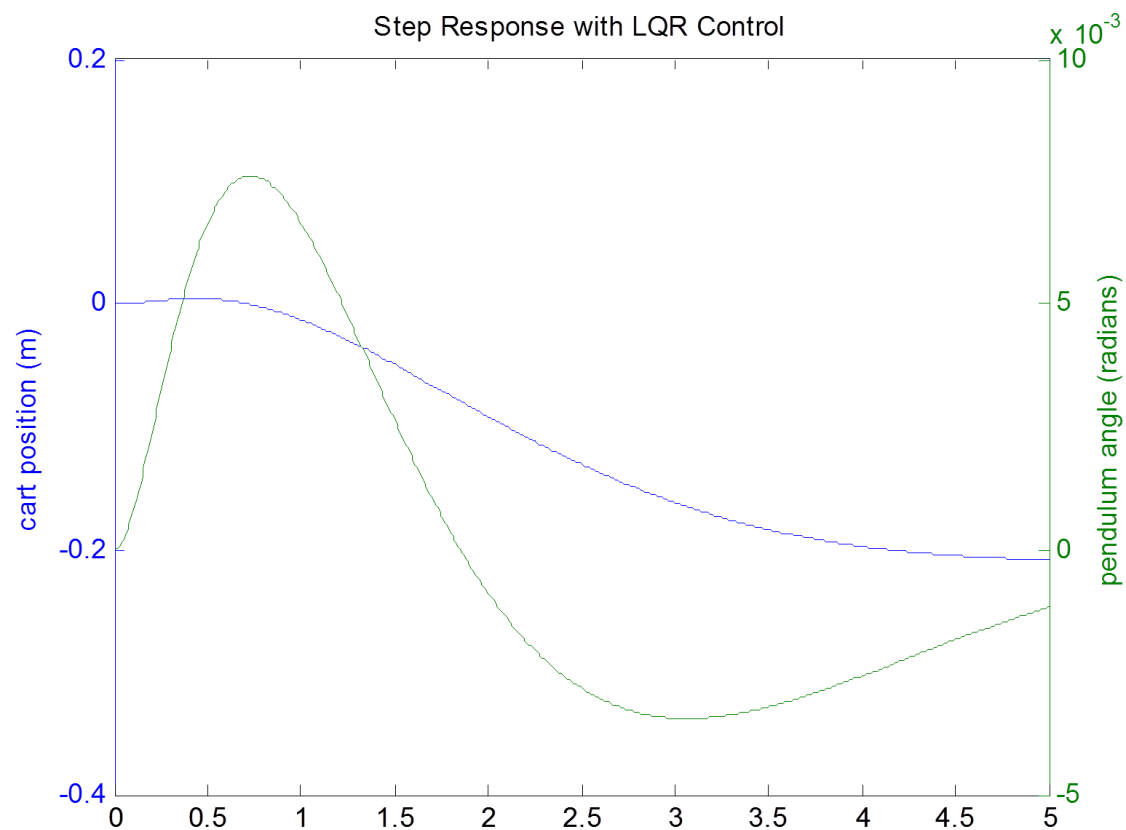
$K =$

-1.0000	-2.1000	32.3890	10.0232
---------	---------	---------	---------

نمودار پاسخ نیز با استفاده از دستورات زیر می توان استخراج شود.

```
Ac=[(A-B*K)];
Bc=[B];
Cc=[C];
Dc=[D];
states={'x','x_dot','phi','phi_dot'};
inputs={'r'};
outputs={'x','phi'};
sys_cl=ss(Ac,Bc,Cc,Dc,'statename',states,'inputname',inputs,'outputname',outputs);
t = 0:0.01:5;
r =0.2*ones(size(t));
[y,t,x]=lsim(sys_cl,r,t);
[AX,H1,H2]=plotyy(t,y(:,1),t,y(:,2),'plot');
set(get(AX(1),'Ylabel'),'String','cart position (m)')
set(get(AX(2),'Ylabel'),'String','pendulum angle (radians)')
title('Step Response with LQR Control')
```

که نتیجه دستور فوق نمودار زیر است.



منحنی سبز رنگ و سمت راست نشان دهنده زاویه آونگ بر حسب رادیان و منحنی آبی رنگ و سمت چپ نشان دهنده موقعیت گاری بر حسب متر است. همان طوری که می بینید این منحنی ها رضایت بخش نیست. حداکثر فرا جهش زاویه آونگ و موقعیت گاری در نمودار به نظر مناسب می رسد ولی زمان نشست نیاز دارد تا بهبود یابد و همچنین زمان صعود گاری باید کاهش یابد. همچنین موقعیت گاری در جهت مخالف حرکت کرده است. به مقدار خطای ماندگار در بخش های بعدی پرداخته می شود و در حال حاضر بر روی زمان صعود و زمان نشست تمرکز می گردد.

با مراجعه به دستورات فوق و تغییر در ماتریس Q به پیدا کردن پاسخ بهتری به صورت آزمایشی می پردازیم. با افزایش مقادیر آرایه های Q و به کاهش زمان نشست، زمان نشست و تغییرات زاویه آونگ منجر می شود. که در واقع این تغییرات وزن خطا را در بخش بهینه سازی دستور lqr افزایش می دهد.

```
Q=C'*C;
Q(1,1)=5000;
Q(3,3)=100;
Q
R=1;
K=lqr(A,B,Q,R)
Ac=[(A-B*K)];
Bc=[B];
Cc=[C];
Dc=[D];
states={'x','x_dot','phi','phi_dot'};
inputs={'r'};
outputs={'x','phi'};
sys_cl=ss(Ac,Bc,Cc,Dc,'statename',states,'inputname',inputs,'outputname',ou
tputs);
t=0:0.01:5;
r=0.2*ones(size(t));
[y,t,x]=lsim(sys_cl,r,t);
[AX,H1,H2]=plotyy(t,y(:,1),t,y(:,2),'plot');
set(get(AX(1),'Ylabel'),'String','cart position (m)')
```

```
set(get(AX(2),'Ylabel'),'String','pendulum angle (radians)')
title('Step Response with LQR Control')
```

با اضافه کردن دستورات فوق به m-file قبلی در نرم افزار MATLAB به نتیجه زیر می‌رسیم.

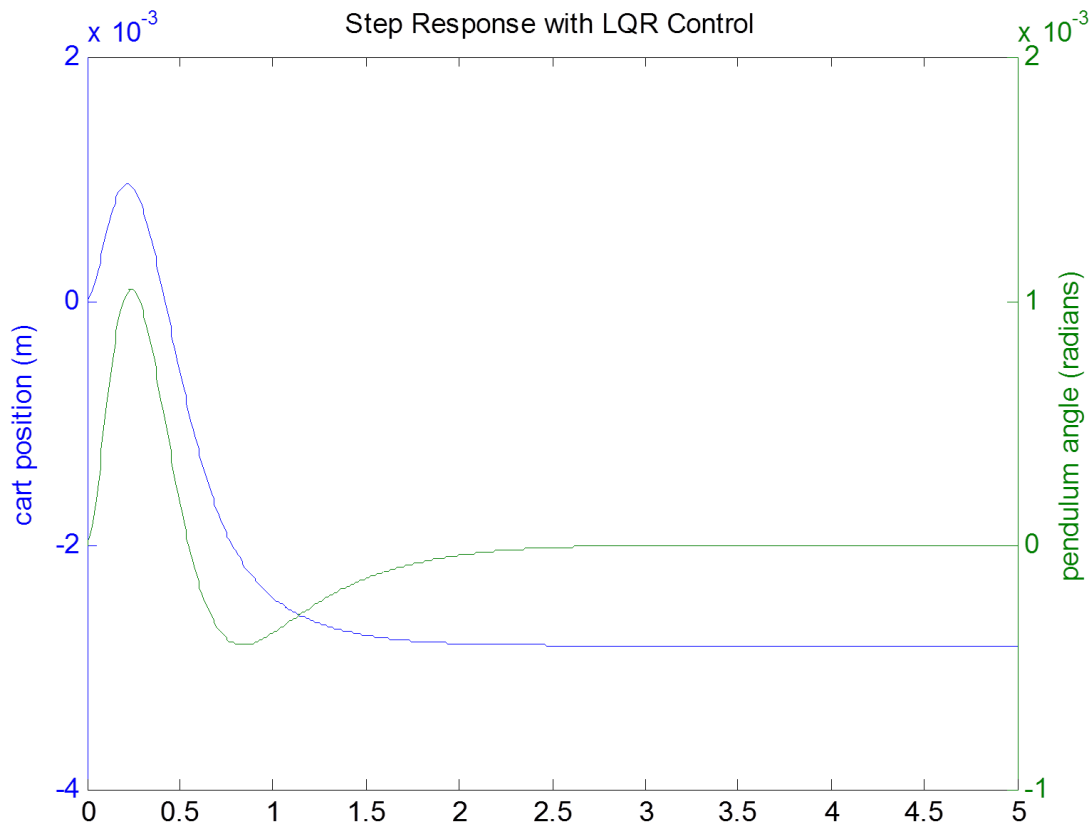
Q =

5000	0	0	0
0	0	0	0
0	0	100	0
0	0	0	0

K =

-70.7107	-57.1791	237.3410	75.4345
----------	----------	----------	---------

و نمودار جدید به صورت زیر می‌گردد.



حتی با افزایش بیشتر مقادیر ماتریس Q به بهبود سیستم بیشتر کمک می‌کند و خطای حالت ماندگار را کاهش می‌دهد. ولی نیاز به نیروی کنترلی بیشتری می‌طلبد. در واقع هزینه ساخت کنترلر، انرژی بیشتر و عملگر بزرگ‌تر را نیاز دارد.

روش استقرار قطب مقاوم place

استقرار قطب مبتنی بر روش استقرار قطب مقاوم یکی از روش‌های مرسوم است که به ماکزیمم کردن حد پایداری می‌انجامد. هرچند فرمان place در نرم افزار MATLAB می‌تواند برای هر دو نوع سیستم یک ورودی و سیستم چند ورودی بکار رود ولی لازمه‌اش این است که چندگانگی قطب در قطب‌های حلقه بسته بیش از مرتبه B نباشد. ماتریس J ماتریسی متشکل از قطب‌های حلقه بسته مطلوب مانند زیر است.

$$J = [\mu_1 | \mu_2 | \dots | \mu_n]$$

که μ_i قطب های حلقه بسته مطلوب است.

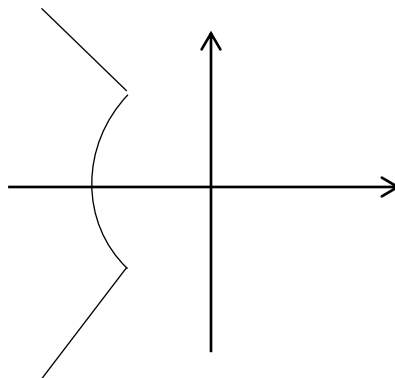
همان طور که گفته شد معیار های طراحی برای انتقال موقعیت گاری x به اندازه 0.2 m به صورت تابع پله به صورت زیر است:

- زمان نشست برای x و θ کمتر از 5 sec باشد.
 - زمان صعود برای x کمتر از 0.5 sec باشد.
 - زاویه انحراف آونگ ϕ نباید بیشتر از 20 deg یا 0.35 rad از حالت عمودی آن باشد.
 - خطای حالت ماندگار کمتر از 2% برای x و θ باشد.
- که با محاسبات زیر می توان محدوده قطب های مطلوب را به دست آورد.

$$\left. \begin{aligned} t_s &= \frac{4.6}{\xi \omega_n} \xrightarrow{t_s < 5} \xi \omega_n > 0.92 \\ t_r &= \frac{1.8}{\omega_n} \xrightarrow{t_r < 0.5} \omega_n > 3.6 \end{aligned} \right\} \rightarrow \begin{cases} \omega_n > 3.6 \\ \xi > 0.26 \end{cases}$$

$$e_{ss} < 0.2\%$$

البته بهتر است $0.5 < \xi < 0.7$ باشد تا پاسخ سیستم سریع تر و با زمان نشست کمتر باشد لذا محدوده قطب های قابل قبول به صورت زیر می گردد.



با استفاده از دستورات زیر می توانیم بردار ضرایب کنترلی K را به دست آوریم.

```
M=1;
m=0.1;
I=0;
g=9.8;
l=1;
p=I*(M+m)+M*m*l^2;
A=[0,1,0,0;0,0,(m^2*g*l^2)/p,0;0,0,0,1;0,0,m*g*l*(M+m)/p,0];
B=[0;(I+m*l^2)/p;0;m*l/p];
C=[1,0,0,0;0,0,1,0];
D=[0;0];
J=[-3.9 -5 -4.4 -4];
K=place(A,B,J)
```

که نتیجه آن ماتریس زیر است.

```
K =
-35.0204 -32.6980 157.6604 49.9980
```

و پاسخ سیستم با استفاده از دستور زیر به دست می آید.

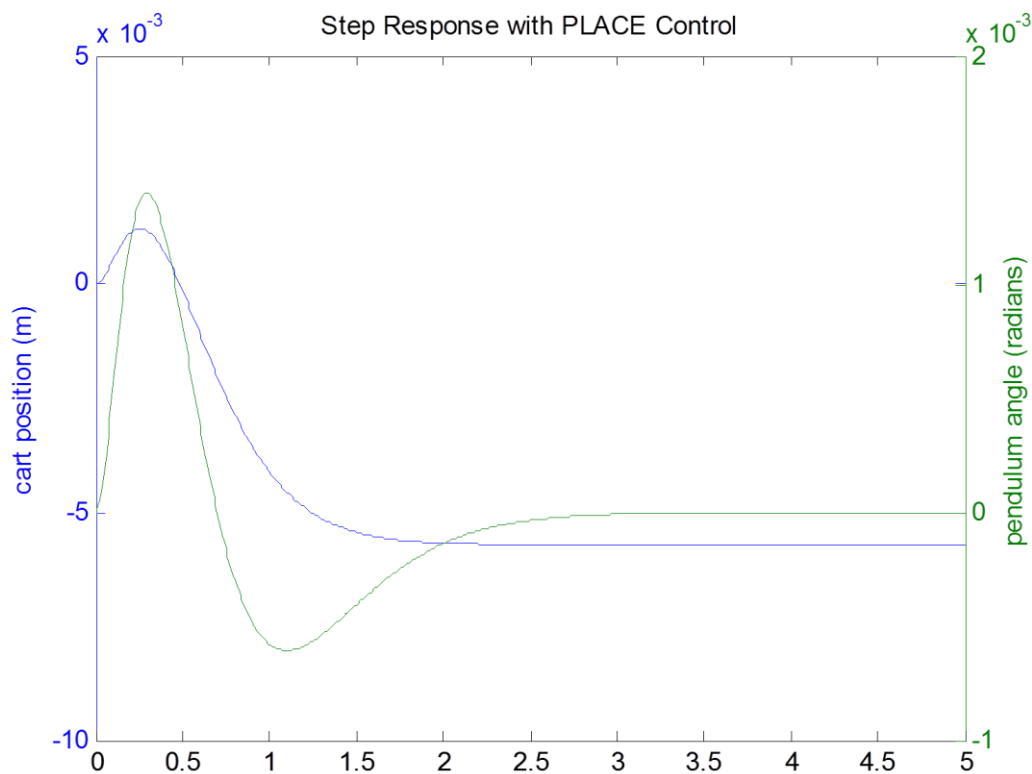
```
Ac=[(A-B*K)];
```

```

Bc=[B];
Cc=[C];
Dc=[D];
states={'x','x_dot','phi','phi_dot'};
inputs={'r'};
outputs={'x';'phi'};
sys_cl=ss(Ac,Bc,Cc,Dc,'statename',states,'inputname',inputs,'outputname',ou
tputs);
t=0:0.01:5;
r=0.2*ones(size(t));
[y,t,x]=lsim(sys_cl,r,t);
[AX,H1,H2]=plotyy(t,y(:,1),t,y(:,2),'plot');
set(get(AX(1),'Ylabel'),'String','cart position (m)')
set(get(AX(2),'Ylabel'),'String','pendulum angle (radians)')
title('Step Response with PLACE Control')

```

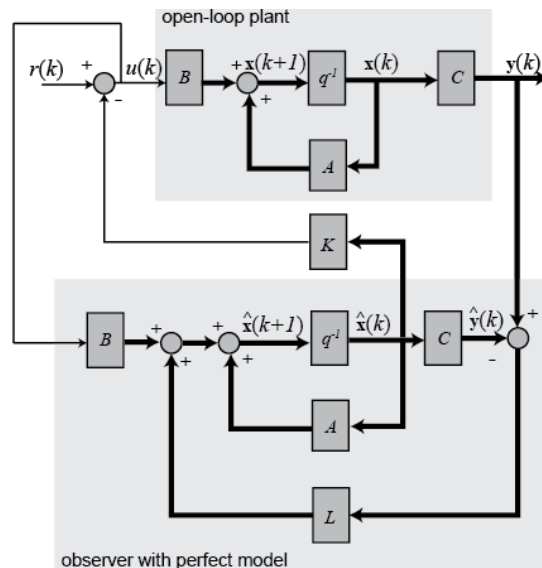
که این دستور نمودار زیر را ارائه می‌دهد.



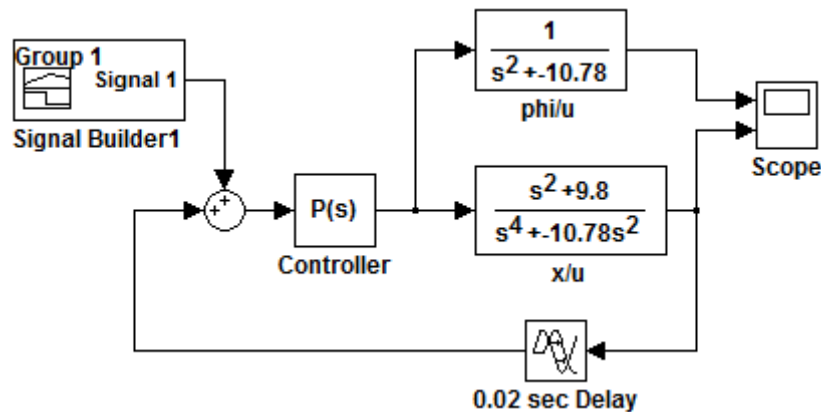
همان طور که از نمودار مشهود است ملاحظات طراحی به خوبی رعایت شده است.

فصل هفتم: طراحی کنترل دیجیتال

قصد داریم سیستم را به صورت دیاگرام زیر کنترل کنیم.



طراحی کنترلر برای سیستم زیر با زمان نمونه برداری ۰.۰۰۵ ثانیه به این صورت است. که ابتدا تابع تأخیر را تقریب می‌زنیم. سپس تابع تبدیل سیستم و تأخیر را پیدا کرده و آن را به فضای حالت برده و شروع به طراحی کنترلر و مشاهده گر می‌کنیم.



تقریب پاده برای دینامیک تأخیر به صورت زیر است.

$$e^{-Ts} = \frac{1-0.5Ts}{1+0.5Ts} \xrightarrow{T=0.02} \frac{1-0.01s}{1+0.01s}$$

حال تابع تبدیل سیستم را به دست می‌آوریم.

```
M=1;
m=0.1;
I=0;
g=9.8;
l=1;
p=I*(M+m)+M*m*l^2;
A=[0,1,0,0;0,0,(m^2*g*l^2)/p,0;0,0,0,1;0,0,m*g*l*(M+m)/p,0];
B=[0;(I+m*l^2)/p;0;m*l/p];
C=[1,0,0,0;0,0,1,0];
```

```
D=[0;0];
states={'x','x_dot','phi','phi_dot'};
inputs={'u'};
outputs={'x','phi'};
sys_ss=ss(A,B,C,D,'statename',states,'inputname',inputs,'outputname',outputs);
[b,a]=ss2tf(A,B,C,D)
```

که نتیجه استفاده از دستور فوق به صورت زیر است.

```
b =
      0      0.0000      1.0000      0.0000     -9.8000
      0      0.0000      1.0000          0          0

a =
      1.0000     -0.0000    -10.7800          0          0
```

که به صورت زیر ساده می شود.

$$\frac{X(s)}{U(s)} = \frac{s^2 - 9.8}{s^4 - 10.78s^2}$$

$$\frac{\phi(s)}{U(s)} = \frac{s^2}{s^4 - 10.78s^2}$$

حال دینامیک تأخیر را در تابع تبدیل مربوط به موقعیت گاری ضرب می کنیم و سیستم را ساده می کنیم.

```
xsysdelay=tf([-0.01,1],[0.01,1]);
xsysplant=tf([1,0,-9.8],[1,0,-10.78,0,0]);
xsys=tf(xsysplant*xsysdelay)
```

که نتیجه استفاده از دستور فوق به صورت زیر است.

```
Transfer function:
      -0.01 s^3 + s^2 + 0.098 s - 9.8
      -----
      0.01 s^5 + s^4 - 0.1078 s^3 - 10.78 s^2
```

البته تابع تبدیل زاویه آونگ به نیرو نیز به صورت زیر تبدیل می شود.

```
phisysd=tf([0.01,1],[0.01,1]);
phisysplant=tf([1,0,0],[1,0,-10.78,0,0]);
xsys=tf(phisysplant*phisysd)
```

که نتیجه دستور فوق به صورت زیر است.

```
Transfer function:
      0.01 s^3 + s^2
      -----
      0.01 s^5 + s^4 - 0.1078 s^3 - 10.78 s^2
```

با استفاده از دستور زیر می توان تابع تبدیل را به فضای حالت تبدیل کرد البته به دلیل اینکه فقط اندازه گیری موقعیت وجود دارد ردیف دوم ماتریس ضرایب صورت صفر است.

```
b=[0,0,-0.01,1,0.098,-9.8;0,0,0,0,0,0];
a=[0.1,1,-0.1078,-10.78,0,0];
[A,B,C,D]=tf2ss(b,a)
```

که نتیجه دستور فوق به صورت زیر است.

```
A =
      0.0000      10.7800          0          0
      1.0000          0          0          0
          0      1.0000          0          0
```

```

0      0      1.0000      0
B =
1
0
0
0
C =
-0.0100      1.0000      98.0000      -9.8000
0      0      0      0
D =
0
0

```

و اگر به صورتی دستی متغیر زاویه آونگ و مشتق زاویه آونگ را به فرمول فوق اضافه کنیم نتیجه به صورت زیر است:

$$\begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\Phi} \\ \ddot{\Phi} \end{bmatrix} = \begin{bmatrix} 0 & 10.78 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \Phi \\ \dot{\Phi} \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} u$$

$$y = \begin{bmatrix} -0.01 & 1 & 98 & -9.8 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \Phi \\ \dot{\Phi} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} u$$

حال این سیستم را به یک سیستم گسسته تبدیل می کنیم.

```

A=[0,10.78,0,0;1,0,0,0;0,1,0,0;0,0,1,0];
B=[1;0;0;0];
C=[-0.01,1,98,-9.8;0,0,0,0];
D=[0;0];
states={'x' 'x_dot' 'phi' 'phi_dot'};
inputs={'u'};
outputs={'x';'phi'};
sys_ss=ss(A,B,C,D,'statename',states,'inputname',inputs,'outputname',outputs);
Ts=5/1000;
sys_d=c2d(sys_ss,Ts,'zoh')

```

که نتیجه دستور فوق در نرم افزار به صورت زیر است.

```

a =
           x      x_dot      phi      phi_dot
x           1      0.0539      0      0
x_dot      0.005      1      0      0
phi        1.25e-005      0.005      1      0
phi_dot    2.083e-008      1.25e-005      0.005      1

b =
           u
x           0.005
x_dot      1.25e-005
phi        2.083e-008
phi_dot    2.604e-011

```

```

c =
      x      x_dot      phi      phi_dot
x      -0.01      1      98      -9.8
phi      0      0      0      0

d =
      u
x      0
phi      0

Sampling time (seconds): 0.005
Discrete-time model.

```

کنترل پذیری و مشاهده پذیری از طریق دستور زیر چک می شود.

```

co = ctrb(sys_d);
ob = obsv(sys_d);
controllability = rank(co)
observability = rank(ob)

```

طبق نتیجه زیر این سیستم کنترل پذیر و مشاهده پذیر است.

```

controllability =

      4

observability =

      4

```

حال با استفاده از دستور زیر به طراحی کنترلر می پردازیم.

```

A = sys_d.a;
B = sys_d.b;
C = sys_d.c;
D = sys_d.d;
Q = C'*C;
R = 1;
[K] = dlqr(A,B,Q,R)
Ac = [(A-B*K)];
Bc = [B];
Cc = [C];
Dc = [D];
states = {'x' 'x_dot' 'phi' 'phi_dot'};
inputs = {'r'};
outputs = {'x'; 'phi'};
sys_cl=ss(Ac,Bc,Cc,Dc,Ts,'statename',states,'inputname',inputs,'outputname',
,outputs);
t = 0:0.005:5;
r =0.2*ones(size(t));
[y,t,x]=lsim(sys_cl,r,t);
[AX,H1,H2] = plotyy(t,y(:,1),t,y(:,2),'plot');
set(get(AX(1),'Ylabel'),'String','cart position (m)')
set(get(AX(2),'Ylabel'),'String','pendulum angle (radians)')
title('Step Response with Digital LQR Control')

```

که نتیجه به صورت زیر است

```

Q =

1.0e+003 *

      0.0000      -0.0000      -0.0010      0.0001
      -0.0000      0.0010      0.0980      -0.0098
      -0.0010      0.0980      9.6040      -0.9604

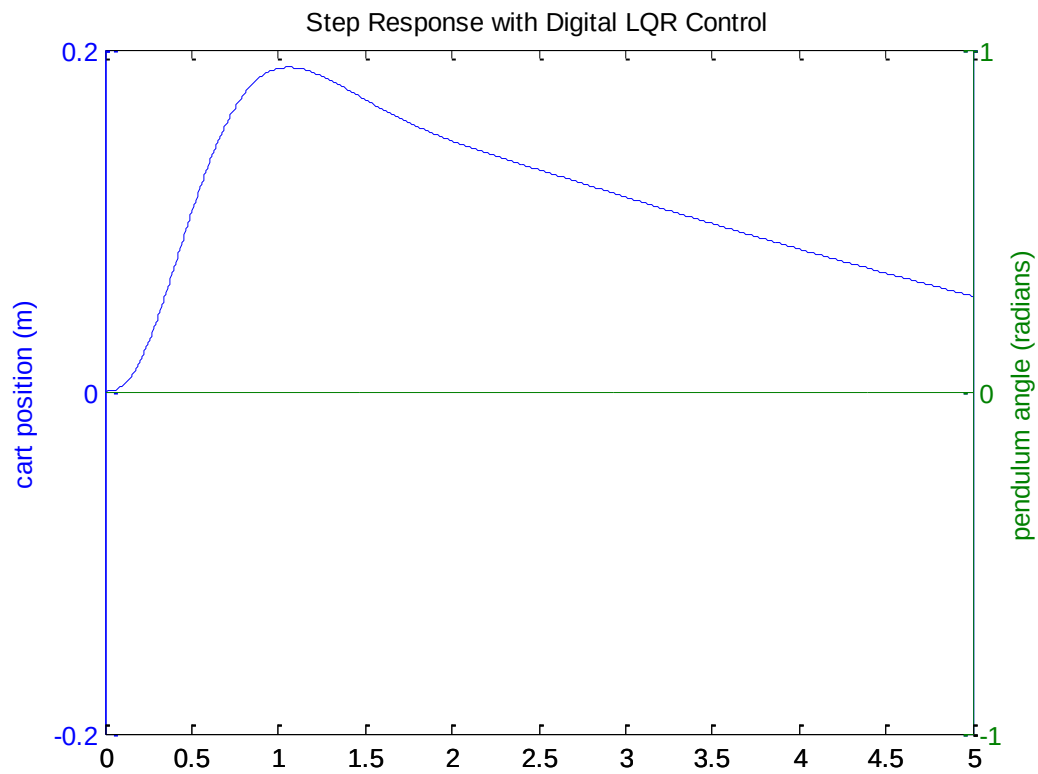
```

```

0.0001    -0.0098    -0.9604    0.0960
K =
10.7264    58.0607    100.0813    9.5372

```

و نمودار پاسخ به صورت زیر است.



طراحی مشاهده گر

قطب‌ها به صورت زیر محاسبه می‌گردد.

```
poles = eig(A-B*K)
```

و نتیجه به صورت زیر است

```

poles =
0.9735
0.9865 + 0.0163i
0.9865 - 0.0163i
0.9995

```

حال با استفاده از این قطب‌ها به محاسبه L از روش Place می‌پردازیم.

```

P = [-0.2 -0.21 -0.22 -0.23];
L = place(A',C',P) '

```

نتیجه به صورت زیر است.

```

L =
1.0e+006 *
0.0019    0
0.0163    0
0.1631    0

```

حال یک مشاهده گر طراحی می کنیم. روش طراحی و دستور های مربوطه به صورت زیر است.

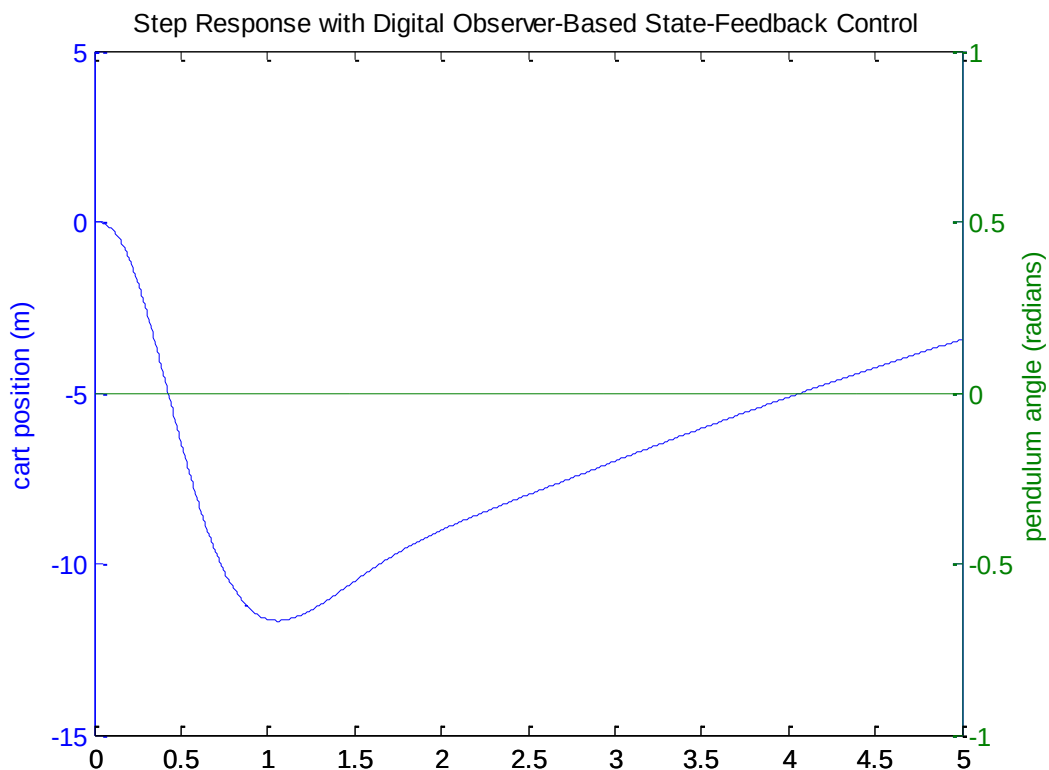
```
Nbar = -61.55;
Ace = [(A-B*K) (B*K);
       zeros(size(A)) (A-L*C)];
Bce = [B*Nbar;
       zeros(size(B))];
Cce = [Cc zeros(size(Cc))];
Dce = [0;0];

states = {'x' 'x_dot' 'phi' 'phi_dot' 'e1' 'e2' 'e3' 'e4'};
inputs = {'r'};
outputs = {'x'; 'phi'};

sys_est_cl =
ss(Ace,Bce,Cce,Dce,Ts,'statename',states,'inputname',inputs,'outputname',ou
tputs);

t = 0:0.005:5;
r = 0.2*ones(size(t));
[y,t,x]=lsim(sys_est_cl,r,t);
[AX,H1,H2] = plotyy(t,y(:,1),t,y(:,2),'plot');
set(get(AX(1),'Ylabel'),'String','cart position (m)')
set(get(AX(2),'Ylabel'),'String','pendulum angle (radians)')
title('Step Response with Digital Observer-Based State-Feedback Control')
```

که پاسخ به تابع پله به صورت زیر است.



فصل نهم: کنترل غیر خطی

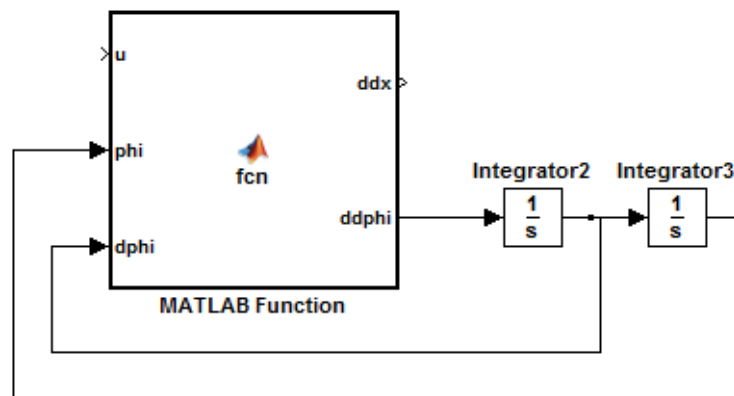
با توجه به استخراج معادلات در فصل اول و بدون خطی سازی معادلات حاکم بر سیستم به صورت زیر در می آیند.

$$\left. \begin{aligned} (M+m)\ddot{x} + ml\ddot{\theta}\cos\theta - ml\dot{\theta}^2\sin\theta &= F \\ (I+ml^2)\ddot{\theta} - mgl\sin\theta &= -ml\ddot{x}\cos\theta \end{aligned} \right\} \rightarrow \begin{aligned} 1.1\ddot{x} + 0.1\ddot{\theta}\cos\theta - 0.1\dot{\theta}^2\sin\theta &= F \\ 0.1\ddot{\theta} - 0.98\sin\theta &= -0.1\ddot{x}\cos\theta \end{aligned}$$

که با ساده سازی داریم:

$$\left. \begin{aligned} 11\ddot{x} + \ddot{\theta}\cos\theta - \dot{\theta}^2\sin\theta - 10F &= 0 \\ \ddot{\theta} - 9.8\sin\theta + \ddot{x}\cos\theta &= 0 \end{aligned} \right\} \rightarrow \begin{aligned} \ddot{\theta} &= 9.8\sin\theta - \frac{\dot{\theta}^2\sin\theta + 10F - 9.8\sin\theta\cos\theta}{11 - \cos^2\theta}\cos\theta \\ \ddot{x} &= \frac{\dot{\theta}^2\sin\theta + 10F - 9.8\sin\theta\cos\theta}{11 - \cos^2\theta} \end{aligned}$$

با ساختن یک MATLAB FUNCTION با معماری زیر می توان با دادن مقدار اولیه ۰.۱ رادیان به انتگرال گیر دوم سیستم شرایط را در خروجی شبیه سازی کرد:



با استفاده بلاک function و المان های مشتق گیر برای شبیه سازی سیستم غیرخطی به صورت فوق از دستور زیر در بلاک استفاده می کنیم.

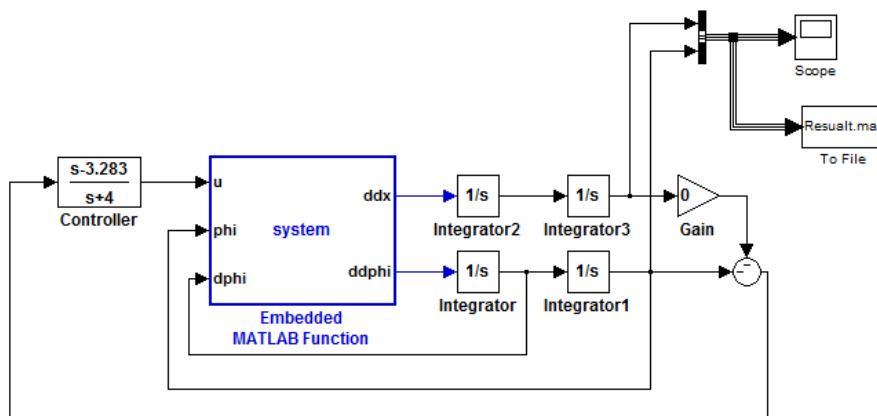
```
function [ddx,ddphi] = fcn(u,phi,dphi)
teta=phi+pi;
dteta=dphi;
ddx=((dteta^2*sin(teta)+10*u-9.8*sin(teta)*cos(teta))/(11-
cos(teta)*cos(teta)));
ddteta=9.8*sin(teta)-((dteta^2*sin(teta)+10*u-9.8*sin(teta)*cos(teta))/(11-
cos(teta)*cos(teta)))*cos(teta);
ddphi=ddteta;
```

پاسخ سیستم غیرخطی با کنترل کلاسیک

از کنترل کننده های طراحی شده در فصل پنجم را یک بار دیگر استفاده می کنیم و نتایج را با همان سیستم ولی به صورت غیرخطی صحت سنجی می کنیم.

کنترل کننده با $K=0$ مربوط به دینامیک سیستم

سیستم را به صورت زیر شبیه سازی می کنیم.



همان طور که در شکل فوق می بینید از یک المان To File استفاده شده است که اطلاعات را ذخیره کند و پس از اجرای برنامه از طریق یک m-file همان اطلاعات را فراخوانی کنیم و نمودار مربوطه را رسم کنیم. این فراخوانی با دستورات زیر انجام می شود.

```
load Result
nt=max(size(ans));
nd=1;
x=ans(1,:);
y1=ans(2,:);
y2=ans(3,:);
plot(x,y1,'b',x,y2,'g');
title('Response with Classic Controller for K=0 Dynamic')
legend('x','phi')
```

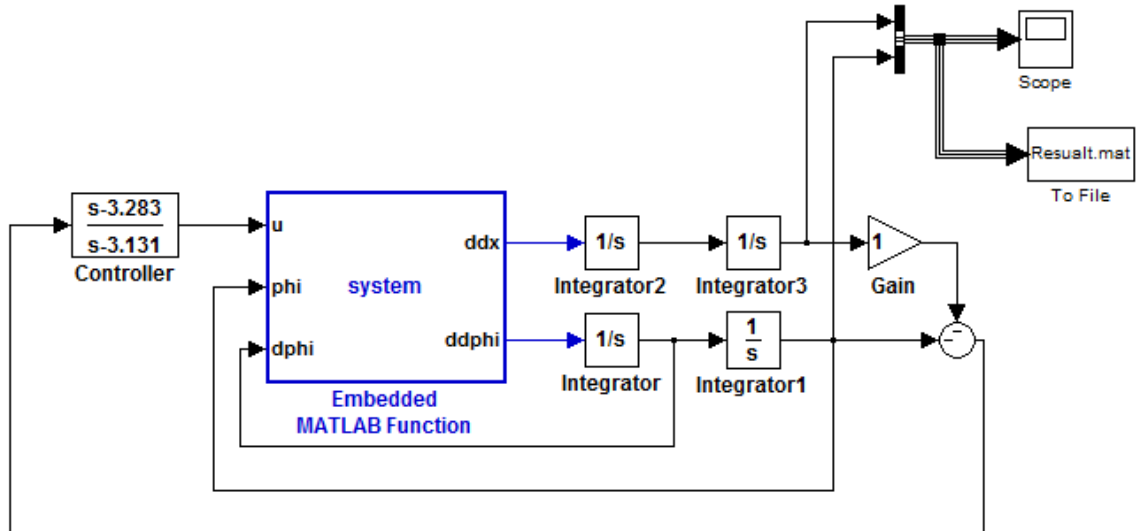
که پاسخ سیستم در ۱۰ ثانیه به صورت زیر است.



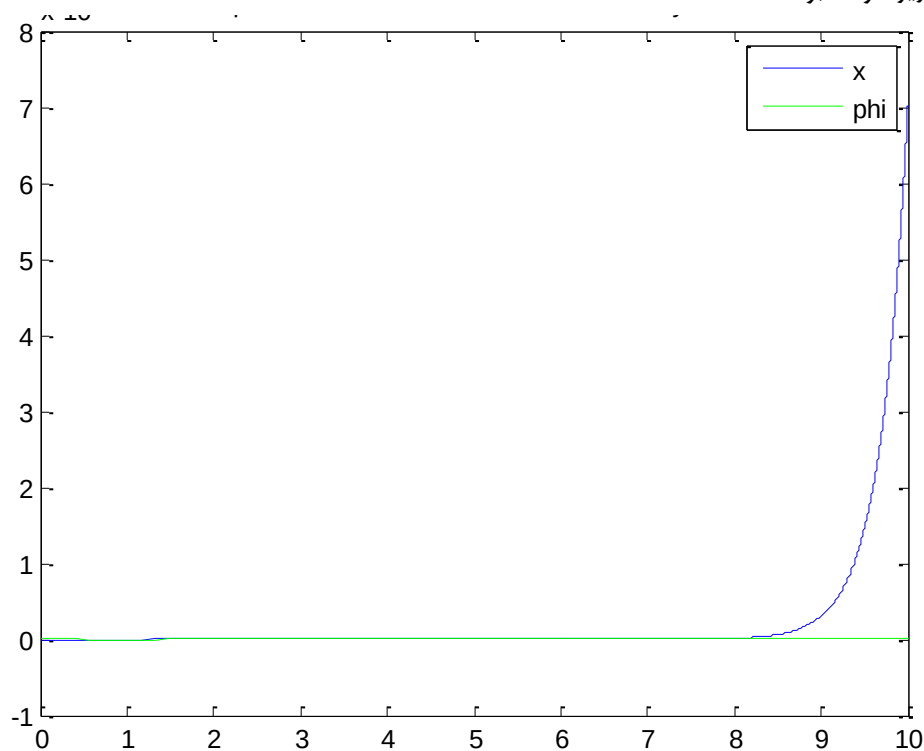
همان طور که گفته شد با طراحی یک جبران ساز نیز می توانیم پاسخ سیستم را بهبود بخشید.

کنترل کننده با $K=1$ مربوط به دینامیک سیستم

این سیستم به صورت زیر شبیه سازی شده است.



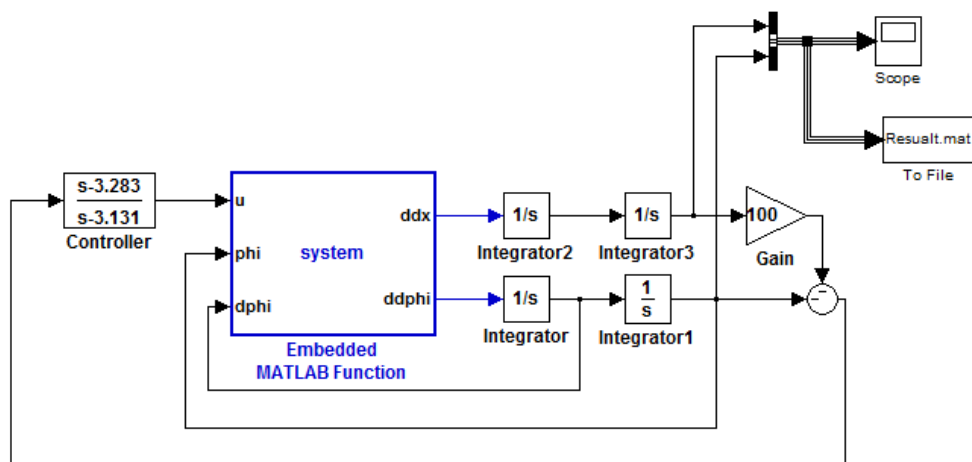
پاسخ به صورت زیر خواهد بود.



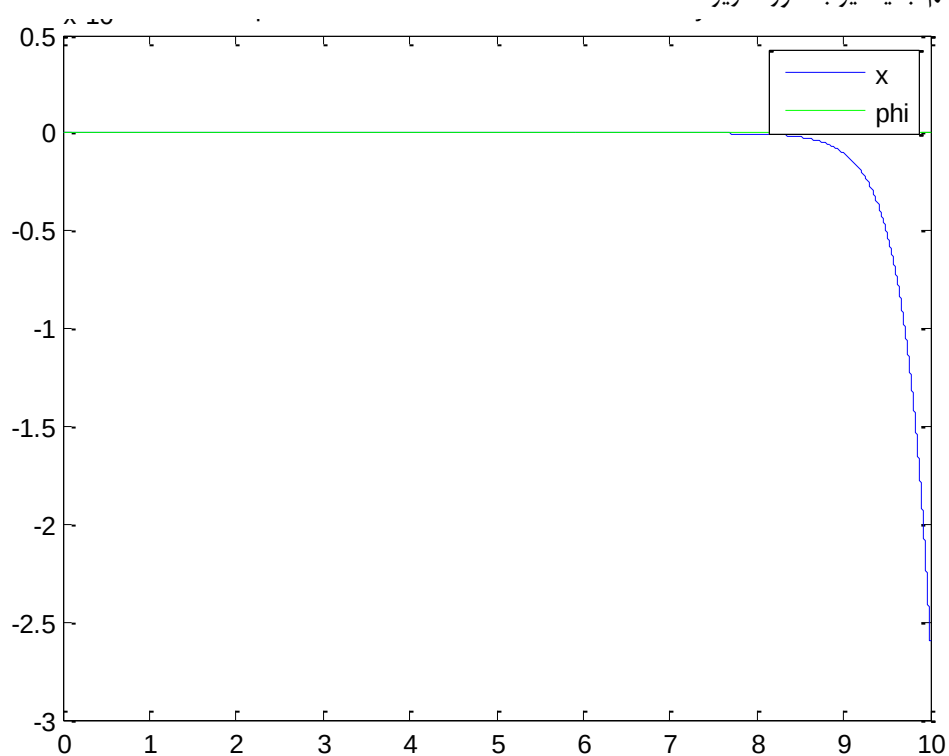
همان طور که متوجه شدید زاویه آونگ توسط کنترل کننده تصحیح شده است ولی موقعیت گاری پس از گذشت زمان کمی ناپایدار شده و افزایش می یابد و این به دلیل عدم طراحی کنترل کننده دقیق است و این گونه طراحی کاملاً وابسته به دینامیک سیستم و با تغییر جزئی آن کنترل کننده پاسخ مناسبی نمی دهد حتی باعث تخریب عملکرد سیستم می شود.

کنترل کننده با $K=100$ مربوط به دینامیک سیستم

این بخش نیز به صورت زیر در نرم افزار MATLAB شبیه سازی می شود.



پاسخ این سیستم جدید نیز به صورت زیر است.

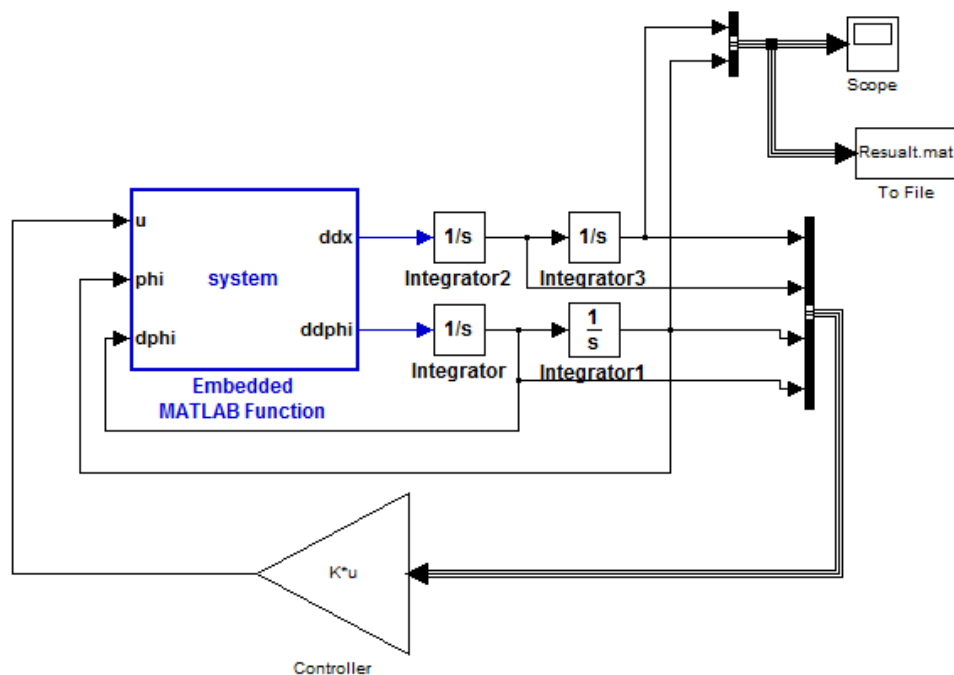


دقیقاً توضیحات قسمت قبل نیز برای این بخش صادق است.

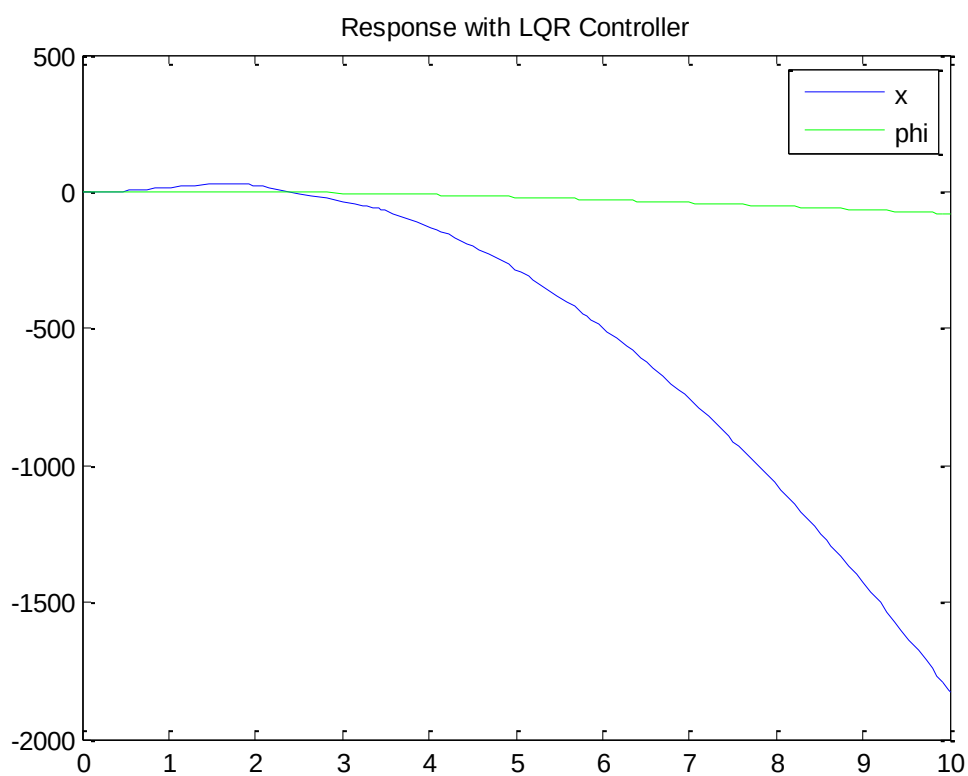
پاسخ سیستم غیر خطی با کنترل کننده مدرن

کنترل کننده طراحی شده با روش مقررات درجه دوم خطی (LQR)

سیستم به صورت زیر در نرم افزار MATLAB پیاده سازی شده است.



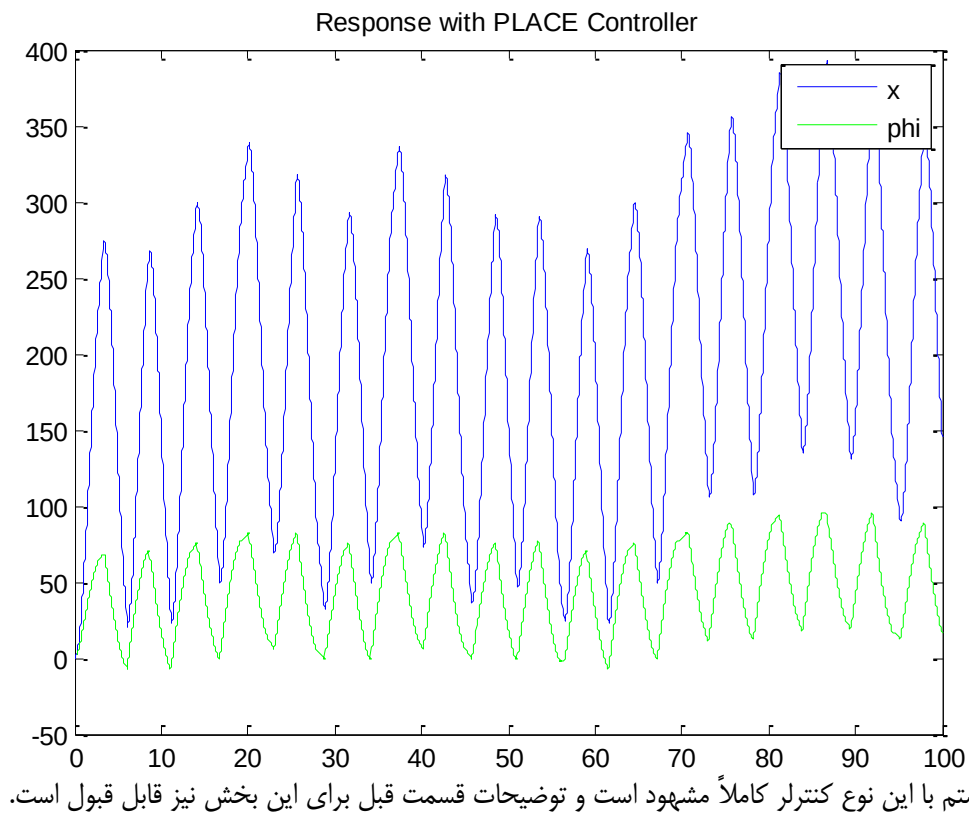
و پاسخ به صورت زیر است.



همان طور که متوجه شدید این پاسخ کاملاً نامطلوب است و تصدیق کننده آن است که سیستم غیرخطی با روش هایی که برای سیستم های LTI کنترل طراحی می شود سازگار نیست.

کنترل کننده طراحی شده با روش استقرار قطب مقاوم place

این بخش نیز مانند بخش قبل شبیه سازی می شود فقط ماتریس K متفاوت است و در فصل ششم بیان شده است. پاسخ این سیستم به صورت زیر است.



فصل دهم: جمع‌بندی

در این فصل با بررسی و مقایسه‌ی فصول گذشته و سؤالات مسئله به نتیجه‌گیری اجمالی از آن چه گذشت می‌پردازیم. نتایج حاصل از حل این مسئله باعث شد که با چالش‌های مختلف برای کنترل یک سیستم *mimo* از جمله برای طراحی نوع کنترلر و معماری‌های آن و مقدار خطای ماندگار و امکان پایدار کردن سیستم در هر نوع طراحی و همچنین تأثیر خطاهای دینامیک داخلی اجزا اندازه‌گیری بر روی کنترل سیستم از جمله سنسورها و پردازش‌کننده‌ها برخوردیم. همچنین با گسسته سازی باعث شد که شبیه‌سازی سیستم خود را باحالت واقعی منطبق تر کنیم. در طراحی کنترلر برای قسمت‌های اول مسئله مشاهده شد با آن که با اعمال ورودی مناسب می‌توان هر دو خروجی را کنترل کرد اما مقداری خطای ماندگار برای یکی از خروجی‌ها داشتیم که غیرقابل اصلاح کلی بود. تأثیر گسسته سازی در دینامیک سیستم را مورد بررسی قرار دادیم. تأثیر خطی سازی در معادلات را با شبیه‌سازی معادلات غیرخطی و مقایسه آن دو با یکدیگر مشاهده نمودیم و نامناسب بودن این نوع کنترل گر عملاً اثبات شد. همچنین کنترل‌پذیری و مشاهده‌پذیری سیستم با داشتن ماتریس‌های مربوطه گویای صحت امکان کنترل سیستم بودند.